

SplitLLM: Collaborative Inference of LLMs for Model Placement and Throughput Optimization

Akrit Mudvari*
Yale University
NEC Laboratories America
akrit.mudvari@yale.edu

Yuang Jiang
NEC Laboratories America
yuangjiang65@gmail.com

Leandros Tassioulas
Yale University
leandros.tassioulas@yale.edu

Abstract—Large language models (LLMs) have been a disruptive innovation in recent years, and they play a crucial role in our daily lives due to their ability to understand and generate human-like text. Their capabilities include natural language understanding, information retrieval and search, translation, chatbots, virtual assistance, and many more. However, it is well known that LLMs are massive in terms of the number of parameters. Additionally, the self-attention mechanism in the underlying architecture of LLMs, Transformers, has quadratic complexity in terms of both computation and memory with respect to the input sequence length. For these reasons, LLM inference is resource-intensive, and thus, the throughput of LLM inference is limited, especially for the longer sequences. In this report, we design a collaborative inference architecture between a server and its clients to alleviate the throughput limit. In this design, we consider the available resources on both sides, i.e., the computation and communication costs. We develop a dynamic programming-based algorithm to optimally allocate computation between the server and the client device to increase the server throughput, while not violating the service level agreement (SLA). We show in the experiments that we are able to efficiently distribute the workload allowing for roughly 1/3 reduction in the server workload, while achieving 19 percent improvement over a greedy method. As a result, we are able to demonstrate that, in an environment with different types of LLM inference requests, the throughput of the server is improved.

Index Terms—Large language models, Collaborative inference, Edge computing

I. INTRODUCTION

LLMs, in recent years, have been playing a progressively more transformative role in our lives by enhancing Natural Language Processing (NLP), exemplified by the Generative Pre-trained Transformers (GPTs) [1]. These language processing tools have served as a backbone for various applications including chatbots, virtual assistants, translation services, and improved search engines. In recent years, it could be said that this technology has played one of the most profound roles in changing how human beings interact with technology.

This fame of LLMs has been primarily driven by general language models, such as GPT-4, which has a tremendous capacity to generate new text based on the aggregated knowledge acquired through training on vast and multidisciplinary data, and Bidirectional Encoder Representations from Transformers (BERT) [2], which has been used for a wide array

of downstream tasks, including text classification. However, there are now various LLMs, which serve a wide array of tasks based on language processing, transcending numerous domains, languages, or specific tasks. For instance, legalBERT [3] is a domain-specific LLM providing legal document analysis, contract review, and more, while BioBERT [4] focuses on biomedical and clinical texts, providing question-answering services within this domain. Other LLMs focus on multilingual and translation services, which include M2M-100 used for providing high-quality translation services without using English as an intermediary, or DeepL Translator [5]. Furthermore, LLMs may be specifically designed for specific tasks. For instance, RoBERTa for NER (Named Entity Recognition), which is a model fine-tuned over RoBERTa, helps identify and classify entities such as names of people, organizations, or locations, from a text. Besides, Wav2Vec [6] is a task-specific tool specializing in converting audio signal to discernible text. Furthermore, there have been numerous visual transformers, such as CMT [7], for image classification.

At the core of these LLMs are the Transformers [8], which are a type of neural network architecture that utilizes multi-headed self-attention mechanism as the core method of recognizing the importance of the relationship between different representations (i.e., different words) in large and complex learning environments. In LLMs, the most popular and widely used application of Transformers, this mechanism captures the relationship between different words in a sentence. Other aspects of this architecture include positional encoders that assign a position-aware identity to the input tokens (parts of each input), and other general Neural Network (NN) structures such as feed-forward NNs. Most of the deep learning models in the past have relied heavily on layers with linear computational and memory complexity with respect to input size; examples include feed forwards NNs, Convolutional Neural Networks (CNNs) or Recurrent Neural Networks (RNNs). However, Transformer models have computational and memory complexity that increase quadratically as the input sequence lengths increase. As we will discuss in further detail in section IV-A, the quadratic complexity means that for longer language inputs, the computation and memory costs grow very quickly. Beyond this, the fact that these LLMs have parameter size ranging from hundreds of millions to tens of billions means that the computation and memory costs are very

*Part of work was done while Akrit Mudvari was an intern at NEC Laboratories America.

SplitLLM: 用于模型放置和吞吐量优化的 LLM 协作推理

阿克里特·穆德瓦里

*

akrit.mudvari@yale.edu

Yale University

NEC Laboratories America

江远

NEC Laboratories America

yuangjian65@gmail.com

Leandros Tassioulas

Yale University

leandros.tassioulas@yale.edu

Abstract近年来, 大语言模型 (LLM) 一直是一项颠覆性创新, 由于它们能够理解和生成类似人类的文本, 因此在我们的日常生活中发挥着至关重要的作用。他们的功能包括自然语言理解、信息检索和搜索、翻译、聊天机器人、虚拟协助等等。然而, 众所周知, 大语言模型的参数数量非常庞大。此外, LLM (Transformers) 底层架构中的自注意力机制在计算和内存方面相对于输入序列长度具有二次复杂度。由于这些原因, LLM 推理是资源密集型的, 因此, LLM 推理的吞吐量受到限制, 特别是对于较长的序列。在本报告中, 我们设计了服务器与其客户端之间的协作推理架构, 以缓解吞吐量限制。在这个设计中, 我们考虑双方的可用资源, 即计算和通信成本。我们开发了一种基于动态编程的算法, 以在服务器和客户端设备之间优化分配计算, 以提高服务器吞吐量, 同时不违反服务级别协议 (SLA)。我们在实验中表明, 我们能够有效地分配工作负载, 从而使服务器工作负载减少大约 1/3, 同时比贪婪方法实现 19% 的改进。因此, 我们能够证明, 在具有不同类型的 LLM 推理请求的环境中, 服务器的吞吐量得到了提高。

Index Terms—大语言模型、协同推理、边缘计算

一、简介

近年来, 大语言模型通过增强自然语言处理 (NLP), 在我们的生活中发挥着越来越大的变革作用, 例如生成式预训练变压器 (GPT) [1]。这些语言处理工具已成为各种应用程序的支柱, 包括聊天机器人、虚拟助理、翻译服务和改进的搜索引擎。近年来, 可以说这项技术在改变人类与技术互动方式方面发挥了最深远的作用之一。

大语言模型的名气主要是由通用语言模型推动的, 例如 GPT-4, 它具有基于通过大量多学科数据训练获得的聚合知识生成新文本的巨大能力, 以及来自 Transformers 的双向编码器表示 (BERT) [2], 它已被广泛用于一系列领域。

下游任务, 包括文本分类。然而, 现在有着各种各样的法学硕士, 它们服务于基于语言处理的广泛任务, 超越了许多领域、语言或特定任务。例如, legalBERT [3] 是特定领域的大语言模型, 提供法律文件分析、合同审查等, 而 BioBERT [4] 专注于生物医学和临床文本, 提供该领域内的问答服务。其他大语言模型专注于多语言和翻译服务, 其中包括用于在不使用英语作为中介的情况下提供高质量翻译服务的 M2M-100, 或 DeepL Translator [5]。此外, 大语言模型可能是专门为特定任务而设计的。例如, 用于 NER (命名实体识别) 的 RoBERTa 是在 RoBERTa 上进行微调的模型, 有助于识别和分类文本中的实体, 例如人名、组织或位置。此外, Wav2Vec [6] 是一种特定于任务的工具, 专门用于将音频信号转换为可识别的文本。此外, 还有许多用于图像分类的视觉转换器, 例如 CMT [7]。

这些 LLM 的核心是 Transformers [8], 它是一种神经网络架构, 利用多头自注意力机制作为识别大型复杂学习环境中不同表示 (即不同单词) 之间关系重要性的核心方法。在大语言模型 (Transformer 最流行且使用最广泛的应用) 中, 这种机制捕获了句子中不同单词之间的关系。该架构的其他方面包括将位置感知身份分配给输入标记 (每个输入的一部分) 的位置编码器, 以及其他通用神经网络 (NN) 结构, 例如前馈 NN。过去大多数深度学习模型都严重依赖于输入大小具有线性计算和内存复杂性的层; 例子包括前馈神经网络、卷积神经网络 (CNN) 或循环神经网络 (RNN)。然而, Transformer 模型的计算和内存复杂性随着输入序列长度的增加呈二次方增加。正如我们将在第 I V-A 节中进一步详细讨论的那样, 二次复杂度意味着对于较长的语言输入, 计算和内存成本增长得非常快。除此之外, 这些 LLM 的参数大小从数亿到数百亿不等, 这意味着计算和内存成本非常高

*Part of work was done while Akrit Mudvari was an intern at NEC Laboratories America.

high even for the LLMs with smaller input sequence lengths.

The Transformer-driven models, while highly costly and only computationally feasible in powerful data center settings in many cases, are nonetheless highly desirable services at the edge of the network. A lot of other non-LLM, AI-driven services are already being deployed at the edge of the network, in Internet of Things (IoT) devices, AR/VR devices, cellphones, and more. This has been facilitated by increased bandwidth, improved latency, and better reliability. Furthermore, network slicing and mobile edge computing are poised to facilitate a wider range of services towards the edge of the network. So, the demand for such models to be deployed for inference is set to grow, and as a result, we can expect an ever-expanding demand for these LLM services at the network edge in the coming years [9]. This will, in turn, put tremendous pressure on the service providers who need to commercially afford the limited capacity under the edge-cloud paradigm. This may be done through another vendor or self-owned servers. Besides the costs, this is set to put pressure on the servers, especially with fluctuating demands leading to throughput issues. The promising capability of these LLMs to provide user-friendly AI for various tasks has been realized, and combined with the idea of multi-modality [10], different types or modules of LLMs may need to be deployed at different locations within the network.

Current efforts for reducing computational loads, especially for the longer sequences in Transformers, focus on reducing the computational complexity of the self-attention layers [11]–[13]. Primarily, this involves approximation of the sparse, low rank, or some combination thereof, of those two approaches. The primary idea is to find computationally and memory-wise cheaper methods of calculating the approximations of the $n \times n$ attention matrix, usually opting for computations that are less than quadratic in complexity. In the sparse approximation approaches [14], the most relevant elements within the matrix are computed while ignoring the parts that are not likely to be as significant. In the low rank approximation approaches, lower-rank representation of the attention matrices are calculated. Some other methods [13] attempt to calculate a combination of those different approaches. While useful, most of these methods fail to reach the accuracy of a full attention matrix, and the ones that do end up approaching the capabilities of the full attention matrices will have lost the computational improvements [15]. While these approaches allow for the utilization of fewer GPU resources and for a shorter time (for either inference or training), it is not always adequate for a vast majority of cases to run most of the LLMs outside the powerful servers, especially if accuracy needs to be optimized.

In recent years, there has been a tremendous growth in facilities that afford computational services under the fog computing paradigm. On the cloud front, there have been numerous efforts leading to improvements in the latency and bandwidth of the services provided by the data centers. For instance, the first half of 2023 saw the largest amount of data center construction in the history [16], and the data center

colocation market is set to reach 131.80 billion dollars, a significant rise from 2022's 57 billion dollars [17]. As the data centers become more prevalent, there is a much better chance of better bandwidth provisioning and lowered latency due to geographical proximity. With the proliferation of 5G cellular networks across the world and the inclusion of Mobile Edge Computing (MEC) as a core technology under 3GPP standardization [18], we can expect to find more servers closer to the user/data. Numerous efforts have been made towards scalable and efficient provisioning of different services under such cellular networks [19]–[21], and many services can benefit from being deployed towards the network edge [22]. The concept of edge computers serving the users closer to the end devices has moved forward significantly in the commercial domains, and the edge data center market, which is already valued at 11.01 billion dollars, is set to grow by 6 times within the next decade [23]. As a result, we will soon find a pervasive network of computational resources, allowing for low latency, and high bandwidth services for AI and other applications. For instance, a user may be able to acquire services from MEC servers close to a base station, as well as closely located powerful data centers with numerous state-of-the-art GPU resources.

The current approach of running LLMs and other large models involves offloading the raw data to a server location where the entire inference is completed. This approach has apparent problems that make it a less than ideal approach, including higher network costs [24], [25], potential throughput loss due to server congestion, and privacy infringement due to offloading of raw input data [26]–[28]. On the other hand, processing data in the local devices, even the ones that are moderately capable, would pose significant issues including a very high task completion latency due to a lack of computational capability. As discussed earlier, this would in effect be significantly worse as the sequence length increases. Besides the discussion earlier, in section IV-C, we will discuss in detail how longer sequence length and self-attention layers make it very difficult to run LLM inference in weaker devices. Hence, a solution in the form of a distributed intelligence approach is split inference [24], [29]. In this approach, certain layers are computed locally on a client device, and then the remaining layers are forwarded to the server devices for further inference. Split Inference (at least layer-wise splitting as we define and discuss in this paper) refers to splitting a neural network into multiple partitions so that different layers of the deep learning model can be computed in different ways. For instance, consider a neural network with layers $l_1, \dots, l_m, \dots, l_n$ where $m < n$; here split learning can involve processing $l_1, \dots, l_m$ locally and then offloading $l_{m+1}, \dots, l_n$ to a server for further computations. Such split learning methods can provide us with various benefits over simply offloading the model as a whole. First, unlike offloading raw data, we can send a processed output, which is able to remain privacy preserving [26]–[28], [30], [31]. Second, the granularity provided by such a division of tasks allows for more optimal scheduling and placement. Third, as evidenced by recent research works [30], [32], it

即使对于输入序列长度较小的大语言模型来说也很高。

Transformer 驱动模型虽，并且在许多情况下只能在强大的数据中心设置中计算可行，但仍然是网络边缘非常理想的服务。许多其他非大语言模型、人工智能驱动的服务已经部署在网络边缘、物联网 (IoT) 设备、AR/VR 设备、手机等中。带宽的增加、延迟的改善和可靠性的提高促进了这一点。此外，网络切片和移动边缘计算有望促进更广泛的网络边缘服务。因此，对部署此类模型进行推理的需求将会增长，因此，我们预计未来几年网络边缘对这些 LLM 服务的需求将不断扩大 [9]。这反过来又会给服务提供商带来巨大压力，他们需要在商业上承担边缘云范式下的有限容量。这可以通过其他供应商或自有服务器来完成。除了成本之外，这还会给服务器带来压力，特别是在需求波动导致吞吐量问题的情况下。这些 LLM 为各种任务提供用户友好的 AI 的希望的能力已经实现，并且结合多模态的思想 [10]，不同类型或模块的 LLM 可能需要部署在网络内的不同位置。

当前减少计算负载的努力，尤其是 Transformers 中较长的序列，重点是减少自注意力层的计算复杂性 [11]-[13]。主要涉及这两种方法的稀疏、低秩或其某种组合的近似。主要想法是找到计算和内存方面更便宜的方法来计算 $n \times n$ 注意力矩阵的近似值，通常选择复杂度小于二次方的计算。在稀疏近似方法 [14] 中，计算矩阵内最相关的元素，同时忽略不太重要的部分。在低秩近似方法中，计算注意力矩阵的较低秩表示。其他一些方法 [13] 尝试计算这些不同方法的组合。虽然这些方法很有用，但大多数都无法达到完整注意力矩阵的准确性，而那些最终接近完整注意力矩阵能力的方法将失去计算上的改进 [15]。虽然这些方法允许利用更少的 GPU 资源和更短的时间（无论是推理还是训练），但对于绝大多数情况来说，在强大的服务器之外运行大多数 LLM 并不总是足够的，特别是在需要优化准确性的情况下。

近年来，在雾计算范式下提供计算服务的设施有了巨大的增长。在云方面，人们做出了大量努力来改善数据中心提供的服务的延迟和带宽。例如，2023 年上半年出现了历史上最大的数据中心建设量 [16]，数据中心

主机托管市场规模预计将达到 1318 亿美元，较 2022 年的 570 亿美元大幅增长 [17]。随着数据中心变得越来越普遍，由于地理位置接近，有更好的机会提供更好的带宽并降低延迟。随着 5G 蜂窝网络在全球范围内的普及以及移动边缘计算 (MEC) 作为 3GPP 标准化的核心技术 [18]，我们可以期望找到更多更接近用户/数据的服务器。为了在这种蜂窝网络下可扩展和高效地提供不同的服务，人们已经做出了许多努力 [19]-[21]，并且许多服务可以从向网络边缘部署中受益 [22]。边缘计算机为更接近终端设备的用户提供服务的概念在商业领域已取得显著进展，边缘数据中心市场价值已达 110.1 亿美元，预计在未来十年内将增长 6 倍 [23]。因此，我们很快就会发现一个无处不在的计算资源网络，为人工智能和其他应用程序提供低延迟和高带宽服务。例如，用户可以从靠近基站的 MEC 服务器以及位置较近、拥有大量最先进 GPU 资源的强大数据中心获取服务。

当前运行 LLM 和其他大型模型的方法涉及将原始数据卸载到完成整个推理的服务器位置。这种方法存在明显的问题，使其不太理想，包括较高的网络成本 [24]、[25]、由于服务器拥塞而导致的潜在吞吐量损失，以及由于卸载原始输入数据而导致的隐私侵犯 [26]-[28]。另一方面，在本地设备中处理数据，即使是能力中等的设备，也会带来重大问题，包括由于缺乏计算能力而导致任务完成延迟非常高。正如前面所讨论的，随着序列长度的增加，这实际上会变得更糟。除了前面的讨论之外，在第 IV-C 节中，我们将详细讨论较长的序列长度和自注意力层如何使得在较弱的设备中运行 LLM 推理变得非常困难。因此，分布式智能方法形式的解决方案是分裂推理 [24]、[29]。在这种方法中，某些层在客户端设备上本地计算，然后将剩余层转发到服务器设备以进行进一步推理。分割推理（至少是我们在本文中定义和讨论的分层分割）是指将神经网络分割成多个分区，以便可以以不同的方式计算深度学习模型的不同层。例如，考虑一个具有层 $l_1, \dots, l_m, \dots, l_n$ 的神经网络，其中 $m < n$ ；这里，分割学习可以涉及在本地处理 $l_1, \dots, l_m$ ，然后将 $l_{m+1}, \dots, l_n$ 卸载到服务器以进行进一步计算。与简单地卸载整个模型相比，这种分割学习方法可以为我们提供多种好处。首先，与卸载原始数据不同，我们可以发送处理后的输出，该输出能够保持隐私保护 [26]-[28]、[30]、[31]。其次，这种任务划分提供的粒度允许更优化的调度和放置。第三，最近的研究工作 [30]、[32] 证明，

could also help with personalization of learned models by keeping the locally trained parts more personalized and having a globally shared set of layers for aggregated learning. And finally, the more important benefit we will explore in this work is an intelligent offloading under the split inference paradigm, where the goal will be to reduce the computational load on the server by optimally delegating certain computational load to the end devices, leading to a throughput improvement for the servers handling large number of AI demand. Layer-wise split learning differs from feature-wise split learning, where in feature-wise split learning, different portions of the input data are distributed across multiple compute nodes for parallel training.

In this work, we develop an intelligent splitting algorithm that will leverage the properties of the LLM models and the input sequence properties (i.e., length) to develop an efficient resource allocation method, which will be optimal under practical assumptions pervasive in today’s fog/edge-cloud computing paradigms. Our goal will be to reduce any computational load (i.e., computational cost in FLOP or GPU memory) in a task-constrained server so that the throughput is improved. The formulation will demand strict user requirements in the form of task completion latency, and we will demonstrate and discuss the effectiveness of this approach for different situations, including for different LLM models and bandwidth availabilities.

II. RELATED WORK

In this section, we discuss different works that are of relevance to our topic and contrast those with the novelty of our work.

Numerous works in literature have focused on scheduling and placement of computational workloads in a distributed setting; however, a multitude of work have also been conducted on extending these problems to an edge-cloud architecture. For instance, in [33], the number of requests is maximized, with storage, computation, and communication costs as constraints, and in [34] similar problem is solved for data-intensive requests. Similarly in [35], a service placement algorithm with a near-optimal solution guarantee is presented. Other works, including [36]–[38], similarly work on different types of placement and scheduling methods for implementing state-of-the-art techniques for efficient allocation of resources under the edge-cloud paradigm. While these works focus on general problems, taking into account the unique characteristics of deep learning architecture can help improve the efficiency of scheduling and placement decisions in multiple ways. For instance, different factors such as energy consumption and network utilization are used in [39] to decide which deep learning models should be run and “where” within a network. In another work, [40], scheduling is developed with approximate models, where some accuracy is traded away for guaranteed service, under constraints such as device energy storage, cloud computing costs and capacity, and execution deadlines. An online variant of the solution is also developed in this work to tackle real-time workloads.

Complementary to scheduling or placement solutions, some methods help reduce the workload while processing deep learning architectures, by employing methods that could be envisioned for different deep learning architectures. Such methods include quantization of the entire network [41]; in the quantization approach, parameters or variables, during inference or backpropagation, can be stored in a way that saves memory and/or computation costs. A much more specific and strict version of quantization is binarization [42], where the model is stored and processed in a binary form. Other methods that aim to reduce the model size include pruning, which includes pruning the parameter space of the model [43]–[45] or pruning the feature space of the model [46]–[48]. These different methods, while effective, inevitably compromise the performance guarantee, since the process often involves sacrificing some model accuracy for an increased compute performance. Even in rare cases where performance may not fall significantly (sometimes, slight pruning may even help tackle over-fitting and increase accuracy for test data), the guarantee of performance cannot be given. Our major contribution in this paper is on developing a method, that not only ensures continued guarantee of optimal performance but also works in a way where these different methods can be complementary to our implementation.

There have been various works in the literature where distributed or split implementations of neural network models aim for collaborative intelligence under different objectives. In [29], the authors proposed a method for collaborative intelligence between end devices and the mobile edge. They divided the model for partial computation at each end, aiming to find the optimal point for splitting the model to improve inference latency and energy efficiency. In [49], the authors develop a method focusing on splitting deep learning architectures at an optimal location to share the model between a client and a server, intending to accelerate the training time, minimizing the effects of bottleneck client devices, and reducing the energy consumption. And in [50], layer-wise split learning is combined with feature-wise splitting to accelerate inference time. A multitude of similar works exist, but the goal of optimization tends to be energy or resource reduction at the client devices with no concern for server throughput, and splitting only takes place at one defined location.

The splitting learning paradigm has since been implemented in various works in literature, including for splitting transformer models for different goals. For instance, in [51], the transformer model for classification and box regression are split and computed separately towards an object detection goal. In [52], the split learning is implemented for a visual transformer with feature extraction and classification done at the client, and the rest of the task offloaded to the server. In [53], a split learning paradigm is implemented, but the neural network architecture itself isn’t split. Here, the prompt phase is run on the client device, and the token generation phase is run on the server. Our work can be considered complementary to this kind of approach, in that we provide further granularity for splitting within a single forward pass. These past works do not

还可以通过使本地训练的部分更加个性化并拥有一组全球共享的聚合学习层来帮助学习模型的个性化。最后，我们在这项工作中将探索的更重要的好处是分割推理范式下的智能卸载，其目标是通过将某些计算负载最佳地委托给终端设备来减少服务器上的计算负载，从而提高处理大量人工智能需求的服务器的吞吐量。分层分割学习与特征分割学习不同，在特征分割学习中，输入数据的不同部分分布在多个计算节点上进行并行训练。

在这项工作中，我们开发了一种智能分割算法，该算法将利用 LLM 模型的属性和输入序列属性（即长度）来开发一种有效的资源分配方法，该方法在当今雾/边缘云计算范式中普遍存在的实际假设下将是最佳的。我们的目标是减少任务受限服务器中的任何计算负载（即 FLOP 或 GPU 内存中的计算成本），从而提高吞吐量。该公式将以任务完成延迟的形式提出严格的用户要求，我们将演示和讨论该方法在不同情况下的有效性，包括不同的 LLM 模型和带宽可用性。

二.相关工作

在本节中，我们讨论与我们的主题相关的不同作品，并将它们与我们作品的新颖性进行对比。

许多文献工作都集中在分布式环境中计算工作负载的调度和放置。然而，为了将这些问题扩展到边缘云架构，也进行了大量工作。例如，在[33]中，以存储、计算和通信成本为约束，请求数量最大化，而在[34]中，针对数据密集型请求解决了类似的问题。类似地，在[35]中，提出了一种具有接近最优解保证的服务放置算法。其他工作，包括[36]-[38]，同样致力于不同类型的放置和调度方法，以实现最先进的技术，以在边缘云范式下有效分配资源。虽然这些工作专注于一般问题，但考虑到深度学习架构的独特特征可以帮助以多种方式提高调度和放置决策的效率。例如，[39]中使用能源消耗和网络利用率等不同因素来决定应该运行哪些深度学习模型以及在网络中的“何处”运行。在另一项工作中，[40]，调度是用近似模型开发的，在设备能量存储、云计算成本和容量以及执行期限等约束下，牺牲一些准确性来换取有保证的服务。这项工作还开发了该解决方案的在线变体，以处理实时工作负载。

作为调度或放置解决方案的补充，一些方法通过采用可为不同深度学习架构设想的方法，有助于减少处理深度学习架构时的工作量。此类方法包括整个网络的量化[41]；在量化方法中，在推理或反向传播期间，参数或变量可以以节省内存和/或计算成本的方式存储。量化的一个更具体和严格的版本是二值化[42]，其中模型以二进制形式存储和处理。其他旨在减小模型大小的方法包括剪枝，其中包括剪枝模型的参数空间[43]-[45]或剪枝模型的特征空间[46]-[48]。这些不同的方法虽然有效，但不可避免地会损害性能保证，因为该过程通常涉及牺牲一些模型精度以提高计算性能。即使在性能可能不会大幅下降的极少数情况下（有时轻微的剪枝甚至可以帮助解决过拟合问题并提高测试数据的准确性），也无法给出性能的保证。我们在本文中的主要贡献是开发一种方法，该方法不仅可以确保持续保证最佳性能，而且可以使这些不同的方法可以与我们的实施相补充。

文献中已经有各种神经网络模型的分布式或分离实现旨在不同目标下的协作智能的作品。在[29]中，作者提出了一种终端设备和移动边缘之间的协作智能方法。他们将模型在每一端进行部分计算，旨在找到分割模型的最佳点，以改善推理延迟和能源效率。在[49]中，作者开发了一种方法，重点是在最佳位置分割深度学习架构，以在客户端和服务端之间共享模型，旨在加快训练时间，最大限度地减少瓶颈客户端设备的影响，并降低能耗。在[50]中，分层分割学习与特征分割相结合，以加速推理时间。存在大量类似的工作，但优化的目标往往是减少客户端设备的能源或资源，而不关心服务器吞吐量，并且分裂仅发生在一个定义的位置。

此后，分裂学习范式已在各种文献著作中得到实现，包括针对不同目标的分裂变压器模型。例如，在[51]中，用于分类和盒回归的变压器模型被分割并分别计算以实现对象检测目标。在[52]中，分割学习是针对视觉转换器实现的，特征提取和分类在客户端完成，其余任务卸载到服务器。在[53]中，实现了分割学习范式，但神经网络架构本身并未分割。这里，提示阶段在客户端设备上运行，令牌生成阶段在服务器上运行。我们的工作可以被认为是这种方法的补充，因为我们为单个前向传播中的分割提供了进一步的粒度。这些过去的作品不

implement decision-making for layer-wise split deep learning models in multiple places, which would allow us to harness the power of high-bandwidth next-generation networks and the varying computational properties of different model layers.

Several studies have aimed to enhance the efficiency of Transformers in particular, especially for processing longer inputs, to reduce the extensively large computation and memory consumption. Longformer [11] combines windowed self-attention and global attention to sparsify the full attention matrix. Similarly, Bigbird [13] introduces a sparse attention method that incorporates random, windowed, and global attention, demonstrating improved performance in tasks such as question answering and summarization. Sparse sinkhorn attention [14] and Reformer [54] incorporate learnable patterns into the attention module. Vyas, Katharopoulos, and Fleuret [55] propose clustered attention, computing attention only for centroids in clustered queries. Other works focus on kernel-based and feature mapping methods, like Performer [56], Reformer [54], and Linformer [57], which improve self-attention efficiency through grouping, clustering, or designing fixed sparse patterns, albeit at the cost of expressiveness. In contrast, our work focuses on a totally different angle: instead of decreasing the computation costs and time for the models at the expense of performance, we consider the split of computation to increase the throughput of LLM inference at the server.

Novelty: In this work, we develop a method of collaborative inference through model splitting for transformer-driven architectures, where the splitting decision is efficient and takes into consideration both the computation and the communication resources. Such an approach makes it suitable for the edge-cloud networks of today and the near future, where significant benefits from resource management could be realized. We develop a method that guarantees optimal results, beyond most of the methods described above, and on top of that, our method is complementary to most of those methods; our method can incorporate various methods that aim to minimize computation or communication costs, and we discuss briefly how our efficient splitting scheme can work alongside aforementioned sparse and low-rank self-attention approximation approaches. We test our method for different LLMs and a visual transformer, comparing it against a greedy implementation. This helps establish the efficacy of our method across different sequence lengths, model types, network environments, and more. Finally, we demonstrate that the method will be useful in improving the throughput at the server that is tasked with providing computation resources to different clients demanding different types of LLM inference services.

This paper is arranged in the following way. In section III, we formulate the split decision-making problem, describe our novel algorithm for intelligent split decision-making, and prove its optimality. In section IV, we begin by demonstrating that efficient splitting can help reduce the computation load at the server without violating the latency requirement. We also show that such efficient splitting schemes are compatible with

other methods of computation load reduction such as sparse and low rank approximation of the self attention matrix. Then we show that our optimal splitting algorithm can outperform a greedy method at reducing the server load, followed by the demonstration that this leads to an improvement in throughput at the servers. We conclude the work in section V.

III. METHOD

In this section, we begin with problem formulation that allows us to establish the relationship between a server and its clients within a communication network, and model the deep learning inference with split/collaborative execution. Within this section, subsection III-A describes the problem formulation that captures the networked infrastructure, alongside the optimization goals. In subsection III-B, the modeled problem is analyzed, computational complexities are discussed, and an optimal algorithm for solving the formulation is presented in subsection III-A. Finally, in subsection III-C, the optimality of the algorithm developed is proven in section III-B.

A. Problem Formulation

Let us consider a network with the server s providing placement service for the end user $e \in E$. Let the model being used for inference be $m \in M$. Our goal is to find the optimal splitting decision $\pi(s, e, m)$, which ensures that a minimal amount of the task is offloaded to the server, while also satisfying a certain latency constraint $\Lambda(e)$, as per the user application requirement. For the connection between the devices e and s , we will define a bandwidth $\delta(s, e)$ to denote the download rate (data going from the device s to device e) and a bandwidth $\delta(e, s)$ to denote the upload rate (data going from the device e to s). This deep learning model can be considered to be made up of layers $l \in L(m)$, with each layer l taking a computation time of $c(e)_l$ to process in device e and $c(s)_l$ to process in device s . For the sake of simplicity during the discussion of the method, we will refer to the added latency of computation for the layer l while moving the task from s to e as c_l . Let the input tensor for each layer be of size τ_l , then the download time can be obtained as $d_l = \tau_l / \delta(s, e)$ and upload time is given by $u_l = \tau_l / \delta(e, s)$.

In a distributed placement scenario, a split neural network model may have certain layers allocated to the server, while the remaining layers are computed locally. The optimal placement policy that aims to minimize the processing load on the server would try to offload certain tasks towards the end device without violating the service level agreement, i.e., the latency requirement of the given application. The decision of whether or not to run a particular layer on the end device is represented by a binary decision variable x_l , where a value of 1 represents that the task can be run in the end device, and a value of 0 represents that the tasks must be run on the server. We will also introduce $l(prev)$ to represent the previous layer of layer l .

在多个地方实现分层分割深度学习模型的决策，这将使我们能够利用高带宽下一代网络的力量和不同模型层的不同计算属性。

一些研究旨在提高 Transformer 的效率，特别是处理较长的输入，以减少大量的计算和内存消耗。Longformer [11] 结合了窗口自注意力和全局注意力来稀疏化完整的注意力矩阵。类似地，Bigbird [13] 引入了一种稀疏注意力方法，该方法结合了随机、窗口和全局注意力，证明了在问答和总结等任务中性能的提高。稀疏沉角注意力[14]和改革者[54]将可学习的模式合并到注意力模块中。Vyas, Katharopoulos 和 Fleuret [55] 提出了集群注意力，仅针对集群查询中的质心计算注意力。其他工作侧重于基于内核和特征映射方法，例如 Performer [56]、Reformer [54] 和 Linformer [57]，它们通过分组、聚类或设计固定稀疏模式来提高自注意力效率，尽管是以表达能力为代价的。相比之下，我们的工作侧重于一个完全不同的角度：我们不会以牺牲性能为代价来降低模型的计算成本和时间，而是考虑分割计算以增加服务器上 LLM 推理的吞吐量。

新颖性：在这项工作中，我们通过变压器驱动架构的模型分割开发了一种协作推理方法，其中分割决策是有效的，并且考虑了计算和通信资源。这种方法使其适用于当今和不久的将来的边缘云网络，可以实现资源管理的显着优势。我们开发了一种保证最佳结果的方法，超越了上述大多数方法，最重要的是，我们的方法是对大多数这些方法的补充；我们的方法可以结合各种旨在最小化计算或通信成本的方法，并且我们简要讨论我们的有效分割方案如何与上述稀疏和低秩自注意力近似方法一起工作。我们针对不同的 LLM 和视觉转换器测试了我们的方法，并将其与贪婪实现进行了比较。这有助于确定我们的方法在不同序列长度、模型类型、网络环境等方面的有效性。最后，我们证明该方法将有助于提高服务器的吞吐量，该服务器的任务是向需要不同类型 LLM 推理服务的不同客户端提供计算资源。

本文的安排如下。在第三节中，我们提出了分割决策问题，描述了我们的智能分割决策新算法，并证明了其最优性。在第四节中，我们首先证明有效的拆分可以帮助减少服务器的计算负载，而不会违反延迟要求。我们还表明，这种有效的分裂方案与

其他减少计算负载的方法，例如自注意力矩阵的稀疏和低秩近似。然后我们证明我们的最佳分割算法在减少服务器负载方面可以优于贪婪方法，然后证明这可以提高服务器的吞吐量。我们在第五节中结束工作。

三. 方法

在本节中，我们从问题表述开始，它使我们能够在通信网络中建立服务器与其客户端之间的关系，并通过拆分/协作执行对深度学习推理进行建模。在本节中，第 III-A 小节描述了捕获网络基础设施以及优化目标的问题表述。在第 III-B 小节中，分析了建模问题，讨论了计算复杂性，并在第 III-A 小节中提出了求解公式的最佳算法。最后，在第 III-C 小节中，所开发算法的最优性在第 III-B 节中得到证明。

A. Problem Formulation

让我们考虑一个网络，其中服务器 s 为最终用户 $e \in E$ 提供放置服务。令用于推理的模型为 $m \in M$ 。我们的目标是找到最佳分割决策 $\pi(s, e, m)$ ，这确保将最少量的任务卸载到服务器，同时根据用户应用程序的要求满足一定的延迟约束 $\Lambda(e)$ 。对于设备 e 和 s 之间的连接，我们将定义带宽 $\delta(s, e)$ 来表示下载速率（从设备 s 到设备 e 的数据）和带宽 $\delta(e, s)$ 来表示上传速率（从设备 e 到 s 的数据）。该深度学习模型可以被视为由层 $l \in L(m)$ 组成，每个层 l 花费 $c(e)_l$ 的计算时间在设备 e 中处理，并在设备 s 中处理 $c(s)_l$ 。为了在讨论该方法时简单起见，我们将任务从 s 移动到 e 时为 l 层增加的计算延迟称为 c_l 。假设每层的输入张量的大小为 τ_l ，则下载时间可以为 $d_l = \tau_l / \delta(s, e)$ ，上传时间为 $u_l = \tau_l / \delta(e, s)$ 。

在分布式放置场景中，分割神经网络模型可能会将某些层分配给服务器，而其余层则在本地计算。旨在最小化服务器上的处理负载的最佳放置策略将尝试将某些任务卸载到终端设备，而不违反服务级别协议，即给定应用程序的延迟要求。是否在终端设备上运行特定层的决策由二元决策变量 x_l 表示，其中值 1 表示任务可以在终端设备上运行，值 0 表示任务必须在服务器上运行。我们还将引入 $l(\text{prev})$ 来表示层 l 的前一层。

Then, the latency constraint for the given model can be defined as:

$$\Lambda(m) \geq \sum_{l \in L(m)} x_l(c(e)_l + (1 - x_{l(\text{prev})})d_l) + \sum_{l \in L(m)} (1 - x_l)(c(s)_l + x_{l(\text{prev})}u_l) \quad (1)$$

In equation 1, the first half of the sum is for situations where the computation for any layer l takes place at the end device, i.e., when x_l is 1, with the computation time being $c(e)_l$, and the download time d_l is considered only when the previous layer was in the server (when $x_{l(\text{prev})} = 1$). The second half is similar but accounts for the situations where the computation takes place at the server.

As long as the task is completed within $\Lambda(m)$, the quality of service agreement is expected to be satisfied, so the goal is then to minimize the number of tasks offloaded to the server. For each layer $l \in L(m)$, we will define the computation load of that layer to be r_l . Then the optimization goal is given by

$$\min_{x_l \forall l \in L(m)} (1 - x_l)r_l \quad (2)$$

r_l is a parameter that may be collected in multiple ways such as by sampling certain metrics during a sample inference. For example, GPU memory usage can be calculated by monitoring the GPU (for instance, we used Nvidia’s system management interface to monitor GPU memory usage). We primarily decided to select the FLOP (Floating Point Operations) for each layer to measure the computation costs. We obtained the FLOP values by calculating them for each layer, and then verified our measurements using an open-source tool, fvcare [58]. The developed algorithm however is designed to work for any layer-wise calculated metric that we aim to minimize.

B. Problem Analysis and Algorithm

Towards solving the aforementioned formulation, the first step is to realize that the problem may not be trivial. Consider the neural network above to be a graph G with each $(\text{layer}, \text{location})$ pair as a node in the graph. A snippet of the equivalent graphical representation of this model inference is illustrated in Figure 1. Running layer l_1 in the end device e is a node (l_1, e) , and (l_1, s) is another node representing running the same layer in server s instead. A link represents the communication between two subsequent layers. For instance, if after layer l_1 is computed in server s , it may be further computed in s as well such that the next hop is (l_2, s) , or it may be downloaded to the client such that the next hop is (l_2, c) . In a case where the computation occurs in the server again or at the server twice in a row, the link between nodes is a tuple of the form $(r_2, k(s)_2)$. Here, the first part of the tuple represents the “weight” of the link, which is the resource expended (which we are trying to minimize), and since the next layer is run in the server, this cost is r_2 . The second element of these link/tuple represents the “cost” of the link, which is how much time was expended in communication, as represented by $k(s)_2$. If the decision was to download

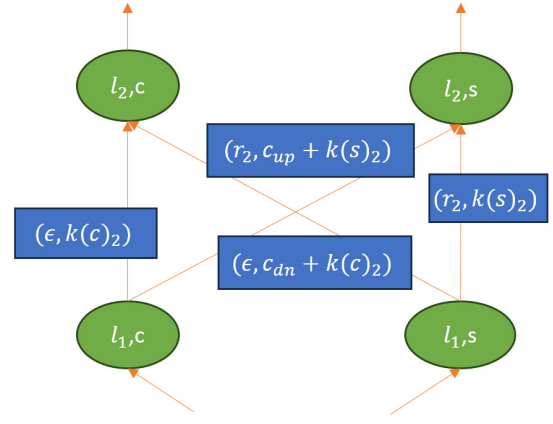


Fig. 1. Illustration of problem as a constrained shortest path problem

instead, the “cost” would entail both compute time at the client ($k(c)_2$) as well as a download cost (k_{dn}), giving a total cost of $k(c)_2 + k_{dn}$. But since we don’t care much about resource usage at the client, this can be represented by a small cost value ϵ . Thus, downloading from the server to the client would give the link a tuple representation $(\epsilon, k(c)_2 + k_{dn})$.

Any inference task must begin with l_1 (starting location may be s or e), and end at l_L (at one of the devices). For instance, suppose we are constrained by the fact that inference starts at (l_1, e) and ends at (l_L, s) . Then our task is to reduce the weight at the server as denoted by equation 2, which means minimizing the computation cost at the server/minimizing traveling through nodes that represent computations in the server, i.e., nodes $(l, s) \forall l \in L$. But, traversing the graph also has a cost constraint: each link in the graph represents a certain time delay, and the sum of these delays as we traverse the links (from start to end node) must not exceed the latency constraint as mentioned in equation 1.

Here, we have reduced the problem formulation from earlier into a constrained shortest path routing problem, which is a class of NP-hard problems [59]. As it stands, this makes it very hard for our optimization goal to be resolved efficiently and on time. However, we are able to develop a pseudopolynomial algorithm to solve this problem by relying on certain relaxations that are unlikely to significantly change the optimal solution. Our solution utilizes an approach where the problem can be solved with a complexity of $\mathcal{O}(Lw)$, where L is the number of distinct layers in the deep learning model, and w is an “integerized” representation of latency limit $\Lambda(m)$. By this we mean that we select w to be an integer value: for instance, for a deadline of 500 milliseconds, it could be 500 with the smallest unit 1 representing 1 millisecond. Without this method, the computational complexity of solving the model would be $\mathcal{O}(2^n)$. As most transformer-driven neural networks, but also most other neural networks, are composed of numerous layers, the complexity makes it very hard to apply a brute force approach to obtain the results quickly.

Our algorithm with a computational complexity of $\mathcal{O}(lw)$ is shown as Algorithm 1. Towards the decision-making, for each

然后，给定模型的延迟约束可以定义为：

$$\Lambda(m) \geq \sum_{l \in L(m)} x_l(c(e)_l + (1 - x_{l(prev)})d_l) + \sum_{l \in L(m)} (1 - x_l)(c(s)_l + x_{l(prev)}u_l) \quad (1)$$

在等式1中，总和的前半部分适用于任意层 l 的计算发生在终端设备的情况，即当 x_l 为1时，计算时间为 $c(e)_l$ ，并且仅当前一层在服务器中时（当 $x_{l(prev)}$ 为1时）才考虑下载时间 d_l 。后半部分类似，但考虑了计算在服务器上进行的情况。

只要任务在 $\Lambda(m)$ 内完成，就有望满足服务质量协议，因此目标是最小化卸载到服务器的任务数量。对于每一层 $l \in L(m)$ ，我们将该层的计算负载定义为 r_l 。那么优化目标由下式给出

$$\min_{x_l \forall l \in L(m)} (1 - x_l)r_l \quad (2)$$

r_l 是可以通过多种方式收集的参数，例如通过在样本推断期间对某些指标进行采样。例如，可以通过监控GPU来计算GPU内存使用情况（例如我们使用Nvidia的系统管理界面来监控GPU内存使用情况）。我们主要决定为每一层选择 FLOP（浮点运算）来衡量计算成本。我们通过计算每一层的 FLOP 值来获得它们，然后使用开源工具 fvcare [58] 验证我们的测量结果。然而，开发的算法旨在适用于我们旨在最小化的任何分层计算指标。

B. Problem Analysis and Algorithm

为了解决上述公式，第一步是认识到问题可能并不微不足道。将上面的神经网络视为一个图 G ，每个 $(layer, location)$ 对作为图中的一个节点。该模型推理的等效图形表示的片段如图1所示。终端设备 e 中运行的层 l_1 是一个节点 (l_1, e) ， (l_1, s) 是另一个节点，表示在服务器 s 中运行同一层。链路表示两个后续层之间的通信。例如，如果在服务器 s 中计算后层 l_1 ，则也可以在 s 中进一步计算，使得下一跳是 (l_2, s) ，或者可以将其下载到客户端，使得下一跳是 (l_2, c) 。在服务器中再次进行计算或连续两次在服务器中进行计算的情况下，节点之间的链接是形式为 $(r_2, k(s)_2)$ 的元组。这里，元组的第一部分代表链接的“权重”，即消耗的资源（我们试图最小化），并且由于下一层在服务器中运行，因此该成本为 r_2 。这些链接/元组的第二个元素表示链接的“成本”，即在通信中花费了多少时间，如 $k(s)_2$ 所示。如果决定下载

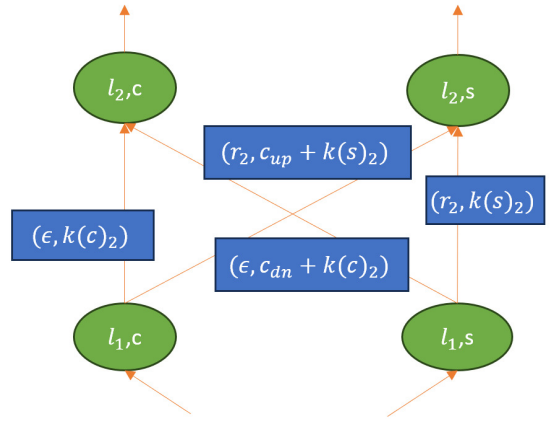


图1. 将问题图示为约束最短路径问题

相反，“成本”需要客户端的计算时间 $k(c)_2$ 以及下载成本 k_{dn} ，总成本为 $k(c)_2 + k_{dn}$ 。但由于我们不太关心客户端的资源使用情况，因此这可以用较小的成本值 ϵ 来表示。因此，从服务器下载到客户端将为链接提供元组表示 $(\epsilon, k(c)_2 + k_{dn})$ 。

任何推理任务都必须以 l_1 （起始位置可以是 s 或 e ）开始，并在 l_L （设备之一）处结束。例如，假设我们受到推理从 (l_1, e) 开始并在 (l_L, s) 结束这一事实的限制。那么我们的任务是减少服务器上的权重，如等式2所示，这意味着最小化服务器上的计算成本/最小化通过表示服务器中计算的节点（即节点 $(l, s) \forall l \in L$ ）的移动。但是，遍历图也有一个成本约束：图中的每个链接都代表一定的时间延迟，并且当我们遍历链接（从起始节点到结束节点）时，这些延迟的总和不得超过等式1中提到的延迟约束。

在这里，我们将之前的问题表述简化为约束最短路径问题，这是一类 NP 难问题 [59]。就目前情况而言，这使得我们的优化目标很难高效、及时地得到解决。然而，我们能够开发一种伪多项式算法来解决这个问题，依靠某些不太可能显著改变最优解的松弛。我们的解决方案采用了一种可以以 $\mathcal{O}(Lw)$ 复杂度解决问题的方法，其中 L 是深度学习模型中不同层的数量， w 是延迟限制 $\Lambda(m)$ 的“整数化”表示。这意味着我们选择 w 为整数值：例如，对于 500 毫秒的截止时间，它可以是 500，最小单位 1 代表 1 毫秒。如果没有这种方法，求解模型的计算复杂度将为 $\mathcal{O}(2^n)$ 。由于大多数变压器驱动的神经网络以及大多数其他神经网络都是由许多层组成的，因此复杂性使得很难应用强力方法来快速获得结果。

我们的计算复杂度为 $\mathcal{O}(lw)$ 的算法如算法1所示。对于决策，对于每个

Algorithm 1 Algorithm for obtaining layer placement policy

Input: For each layer $l \in L$, client inference time $i_l \in \hat{I}$, input download time $d_l \in \hat{D}$, input upload time $u_l \in \hat{U}$, and computation resource usage/cost r_l . Inference deadline Λ , Smallest time unit T , start-at-client flag SaC

Output: Layer schedule policy π

```
1: Initialize: Layer schedule Vector  $\pi$  of size  $|L|$  with  $\pi_l = 0$ 
    $\forall l \in \{0, \dots, |L|\}$ 
2: Obtain: "Integer Equivalent" for every time-related inputs
   with the function  $I, D, U, W = \text{Inteq}(\hat{I}, \hat{D}, \hat{U}, \Lambda, T)$ 
   (algorithm 2)
3: Initialize: Storage matrices  $C$  and  $S$  of size  $(|L| + 1, W + 1)$ 
   with  $C(k, j) = 0$  and  $S(k, j) = 0 \forall k \in \{0, \dots, |L| + 1\}$ 
   and  $\forall j \in \{0, \dots, W + 1\}$ 
4: Initialize:  $\epsilon = -W$ 
5: Trivial case: first row of  $C$  and  $S$ 
6: for rows  $k = 2, \dots, |L| + 1$  do
7:   for columns  $j = 1, \dots, W$  do
8:     if  $j - u_k < 0$  then
9:       Set:  $c2s = \epsilon$ 
10:    else
11:      Set:  $c2s = j - u_k$ 
12:    end if
13:    Set:  $c2c = \max(0, j - i_k)$ 
14:    Set:  $s2c = \max(0, j - i_k - d_k)$ 
15:    Set:  $C(k, j) = \max(C(k - 1, c2c), S(k - 1, s2c))$ 
16:    Set:  $S(k, j) = \max(C(k - 1, c2s), S(k - 1, j))$ 
17:  end for
18: end for
19: Initialize:  $w = W$  to represent time resource remaining
20: for entry  $k = |L|, \dots, 1$  do
21:   Set:  $c2c = \max(0, w - i_k)$ 
22:   Set:  $c2s = \max(0, w - u_k)$ 
23:   Set:  $s2c = \max(0, w - i_k - d_k)$ 
24:   Set:  $s2c = 0$ 
25:   if  $\pi[k + 1] == 0$  then
26:     if  $S(k, s2s) < C(k, s2c)$  then
27:       Set:  $\pi[k] = 1$ 
28:       Set:  $w = w - s2c$ 
29:     else
30:       Set:  $w = w - s2s$ 
31:     end if
32:   else
33:     if  $S(k, c2s) < C(k, c2c)$  then
34:       Set:  $\pi[k] = 1$ 
35:       Set:  $w = w - c2c$ 
36:     else
37:       Set:  $w = w - s2c$ 
38:     end if
39:   end if
40: end for
```

layer $l \in L$, model information to be provided are client's

Algorithm 2 Algorithm for obtaining integer approximation (Inteq function)

Input: For each layer $l \in L$, client inference time $i_l \in \hat{I}$, input download time $d_l \in \hat{D}$, input upload time $u_l \in \hat{U}$, inference deadline Λ , and smallest time unit T

Output: Integer approximation W, I, D, R for inputs $\hat{I}, \hat{D}, \hat{U}, \Lambda$

```
1: for rows  $k = 1, \dots, |L|$  do
2:   Set:  $i_k = \text{round}(\hat{i}_k/w)$ 
3:   Set:  $d_k = \text{round}(\hat{d}_k/w)$ 
4:   Set:  $u_k = \text{round}(\hat{u}_k/w)$ 
5: end for
6: Set:  $W = \text{round}(\Lambda/w)$ 
```

inference time $i_l \in \hat{I}$, download time for the output of the previous layer from server to client $d_l \in \hat{D}$, upload time for the output of the previous layer from client to server $u_l \in \hat{U}$, and resource cost as explained earlier r_l . other non-layer specific information to be provided are the inference deadline for the model Λ , and the smallest time unit T . Since our algorithm requires the aforementioned time units to be treated as integers, T is a variable that decides the smallest possible unit w . It was seen that a very small value of T was still sufficient to run the algorithm in real time, and this value can effectively be 1 ms or even lower. The algorithm will return as output the layer scheduling policy π of size $|L|$. This is a binary vector with $\pi_l = 0$ if the algorithm decides that the layer should be run in the server, and $\pi_l = 1$ if that part should run locally.

In line 2 of algorithm 1, the information $\hat{I}, \hat{D}, \hat{U}, \Lambda, T$ are taken as input and using algorithm 2 (also called Inteq), the integer equivalent I, D, U, W as discussed earlier is obtained. The dynamic programming aspect of the algorithm starts at line 3 with the creation of matrices C and S of size $(|L| + 1, W + 1)$ and all values initialized to zero. The two tables are dynamically updated with $C(l, w)$ referring to optimal placement up to capacity $w \in W$ for up to layer l at the client with l_{th} layer remaining at the client, and $S(l, w)$ referring to optimal placement up to capacity $w \in W$ for up to layer l at the server with the l_{th} layer remaining at the server. The lowest possible value, ϵ needs to be sufficiently small, and this value can be less than $-W$.

Lines 6-18 in algorithm 1 refer to the population of the tables C and S , which will be used by the selection part of the algorithm (lines 19 onward) for returning the optimal placement decision variable π . An explanation of how this section works would benefit from looking at an arbitrary layer l and weight w . At that point (l, w) , 4 different actions are possible: server-to-client $s2c$ (where the previous layer was computed on the server and the current will be computed on the client), server-to-server $s2s$ (both previous and current layers are computed on the server), client-to-client $c2c$ (both previous and current layers are computed on the client), and client-to-server $c2s$ (where the previous layer was computed

Algorithm 1 Algorithm for obtaining layer placement policy

Input: For each layer $l \in L$, client inference time $i_l \in \hat{I}$, input download time $d_l \in \hat{D}$, input upload time $u_l \in \hat{U}$, and computation resource usage/cost r_l . Inference deadline Λ , Smallest time unit T , start-at-client flag SaC

Output: Layer schedule policy π

```

1: Initialize: Layer schedule Vector  $\pi$  of size  $|L|$  with  $\pi_l = 0$ 
    $\forall l \in \{0, \dots, |L|\}$ 
2: Obtain: "Integer Equivalent" for every time-related inputs
   with the function  $I, D, U, W = \text{Inteq}(\hat{I}, \hat{D}, \hat{U}, \Lambda, T)$ 
   (algorithm 2)
3: Initialize: Storage matrices  $C$  and  $S$  of size  $(|L| + 1, W + 1)$ 
   with  $C(k, j) = 0$  and  $S(k, j) = 0 \forall k \in \{0, \dots, |L| + 1\}$ 
   and  $\forall j \in \{0, \dots, W + 1\}$ 
4: Initialize:  $\epsilon = -W$ 
5: Trivial case: first row of  $C$  and  $S$ 
6: for rows  $k = 2, \dots, |L| + 1$  do
7:   for columns  $j = 1, \dots, W$  do
8:     if  $j - u_k < 0$  then
9:       Set:  $c2s = \epsilon$ 
10:    else
11:      Set:  $c2s = j - u_k$ 
12:    end if
13:    Set:  $c2c = \max(0, j - i_k)$ 
14:    Set:  $s2c = \max(0, j - i_k - d_k)$ 
15:    Set:  $C(k, j) = \max(C(k - 1, c2c), S(k - 1, s2c))$ 
16:    Set:  $S(k, j) = \max(C(k - 1, c2s), S(k - 1, j))$ 
17:  end for
18: end for
19: Initialize:  $w = W$  to represent time resource remaining
20: for entry  $k = |L|, \dots, 1$  do
21:   Set:  $c2c = \max(0, w - i_k)$ 
22:   Set:  $c2s = \max(0, w - u_k)$ 
23:   Set:  $s2c = \max(0, w - i_k - d_k)$ 
24:   Set:  $s2c = 0$ 
25:   if  $\pi[k + 1] == 0$  then
26:     if  $S(k, s2s) < C(k, s2c)$  then
27:       Set:  $\pi[k] = 1$ 
28:       Set:  $w = w - s2c$ 
29:     else
30:       Set:  $w = w - s2s$ 
31:     end if
32:   else
33:     if  $S(k, c2s) < C(k, c2c)$  then
34:       Set:  $\pi[k] = 1$ 
35:       Set:  $w = w - c2c$ 
36:     else
37:       Set:  $w = w - s2c$ 
38:     end if
39:   end if
40: end for

```

第 $l \in L$ 层, 提供的模型信息是客户端的

Algorithm 2 Algorithm for obtaining integer approximation (Inteq function)

Input: For each layer $l \in L$, client inference time $i_l \in \hat{I}$, input download time $d_l \in \hat{D}$, input upload time $u_l \in \hat{U}$, inference deadline Λ , and smallest time unit T

Output: Integer approximation W, I, D, R for inputs $\hat{I}, \hat{D}, \hat{U}, \Lambda$

```

1: for rows  $k = 1, \dots, |L|$  do
2:   Set:  $i_k = \text{round}(\hat{i}_k/w)$ 
3:   Set:  $d_k = \text{round}(\hat{d}_k/w)$ 
4:   Set:  $u_k = \text{round}(\hat{u}_k/w)$ 
5: end for
6: Set:  $W = \text{round}(\Lambda/w)$ 

```

推理时间 $i_l \in \hat{I}$ 、从服务器到客户端的上一层输出的下载时间 $d_l \in \hat{D}$ 、从客户端到服务器的上一层输出的上传时间 $u_l \in \hat{U}$ 以及前面解释的资源成本 r_l 。要提供的其他非层特定信息是模型的推理期限 Λ 和最小时间单位 T 。由于我们的算法要求将上述时间单位视为整数, 因此 T 是决定最小可能单位 w 的变量。可以看出, 非常小的 T 值仍然足以实时运行算法, 并且该值实际上可以是 1 ms 甚至更低。该算法将返回大小为 $|L|$ 的层调度策略 π 作为输出。这是一个二进制向量, 如果算法决定该层应在服务器中运行, 则 $\pi_l = 0$; 如果该部分应在本地运行, 则为 $\pi_l = 1$ 。

在算法 1 的第 2 行中, 将信息 $\hat{I}, \hat{D}, \hat{U}, \Lambda, T$ 作为输入, 并使用算法 2 (也称为 Inteq), 获得前面讨论的整数等效项 I, D, U, W 。该算法的动态编程方面从第 3 行开始, 创建大小为 $(|L| + 1, W + 1)$ 的矩阵 C 和 S , 并将所有值初始化为零。这两个表会动态更新, 其中 $C(l, w)$ 指的是客户端上高达 l 层的容量为 $w \in W$ 的最佳放置, 而 l_{th} 层保留在客户端, 而 $S(l, w)$ 指的是服务器上高达 l 层的容量为 $w \in W$ 的最佳放置, 而 l_{th} 层保留在服务器上。最低可能值 ϵ 需要足够小, 并且该值可以小于 $-W$ 。

算法 1 中的第 6-18 行引用表 C 和 S 的总体, 算法的选择部分 (第 19 行开始) 将使用它们来返回最佳布局决策变量 π 。查看任意层 l 和权重 w 将有助于解释本节的工作原理。此时 (l, w) , 可能有 4 种不同的操作: 服务器到客户端 $s2c$ (其中前一层在服务器上计算, 当前层将在客户端上计算), 服务器到服务器 $s2s$ (前一层和当前层都在服务器上计算), 客户端到客户端 $c2c$ (前一层和当前层都在客户端上计算), 以及客户端到服务器 $c2s$ (其中计算前一层

Table: Client(C)		Capacity $\rightarrow$		
Layers $\rightarrow$		$w = m$	$w = m + 1$	...
				...
	l_n	$\max(s2c, c2c)$	$\max(s2c, c2c)$	...
	l_{n+1}	$\max(s2c, c2c)$	$\max(s2c, c2c)$	...
	$\vdots$	$\vdots$	...	...

Table: Server(S)		Capacity $\rightarrow$		
		$w = m$	$w = m + 1$	...
				...
	l_n	0	Run at server	Run at server

Fig. 2. Illustration of dynamic allocation tables as described in Algorithm 1

on the client and the current layer will be computed on the server). The time cost associated with each of those configurations is calculated accordingly: for instance, $c2c$ would only have the cost of computing in the k^{th} layer, since staying in the same device does not incur any communication cost.

It is necessary to ensure that the costs for the j^{th} column of the matrices C and S only consider feasible actions, so the cases that will cause these costs to exceed j will only have a value of ϵ . For instance, $s2c$ will need to have a total cost value of $i_k + d_k$ (for downloading to the client and then processing in the client), and this value should be less than or equal to the availability value j . Also, line 8 ensures that enough budget is always available, to ensure that there are enough resources to upload to the server; this is done by assigning a very small value ϵ to $c2s$ when uploading is infeasible. The algorithm automatically ignores policies that have such large negative values. Finally, once all the associated costs are calculated for up to a given j and k , then values of $C(k, j)$ and $S(k, j)$ are now available, and these values help determine the cost associated with having k layers under cost/time budget j , at client or server device respectively. The tables, starting from the top-left are filled for each row first, left to right, before moving to the row below, as suggested by lines 6-7, until the entire table is filled.

Figure 2 intuitively shows the process of assigning values to the table. For instance, let us assume the task were optimal/assigned at l_{n-1} in the server S (Just to illustrate what happens when optimal assignment is in S at l_n). Then the options are for the tasks to be offloaded to the client C for processing at layer l_{n+1} , with the cost $s2c$ described earlier, or continue running in the server. For the client C , the optimal decision at layer l_{n+1} is the better option between fetching data and processing the output of l_n at S , or continuing with the policy of running at client, hence $\max(s2c, c2c)$.

The second part of algorithm 1, lines 19-39, are for using the now populated tables C and S to discern the best policy π , used for assigning each layer to either the server or the client.

We begin by assigning the value W to a variable w , which will keep track of how much time resource has been spent. The goal is to keep track of integerized variable w ($0 \leq w \leq W$) that has been assigned to the layers for which a decision has already been made. We begin iterating bottom-to-top across matrices C and S , starting with the decision for the last layer and ending with the first. For any layer k , the decision to whether or not to keep the task at the client is decided by the entry in $\pi[k]$. The first step (lines 21-24) within this part of the algorithm is for deciding what was the cost of reaching the selected state (either reaching client or server) for k layer is, given the different costs associated with the processing layer and transferring the layer output if upload/download is needed. Take line 21 assuming $w > i_k$, where w represents the time resource remaining and i_k is the cost of processing the data in the client device in i^{th} layer. The value of $c2c$ then represents how much budget remains after the decision to run k^{th} layer at the client device uses up, given that k^{th} layer was run on the client as well. Similarly, $c2s$ represents the same for when k^{th} layer was run on the client and $k+1^{th}$ on the server. $s2c$ and $c2s$ are calculated similarly.

The next step is to determine the allocation of k^{th} layer given the values of $k+1^{th}$ layer, and the associated costs of processing and transferring. If $k+1^{th}$ layer was to be processed in the client as decided by the decision variable $\pi[k+1]$ being 0 (line 25), then the next step would be to compare the costs to determine whether the given step would be optimally allocated at the server ($\pi[k] = 1$) or the client ($\pi[k] = 0$). This decision is made by comparing the values at $S(k, s2s)$ and $C(k, s2c)$ to determine which of the two allocations would have left the most resources for the future layers. After a decision is made, it is important to remove the resources used up by the k^{th} layer. For instance, in line 28, by which point it will have been decided that $k+1^{th}$ layer was going to be processed in the server, and k^{th} layer was going to be run in the client. Then, the new weight is given by $w - s2c$, where $s2c$ is the cost running k^{th} layer on the client given by i_k added to the cost of sending data from client to server d_k .

Upon the completion of algorithm 1, the optimal placement policy π is obtained.

C. Proof of Optimality

While the problem formulation was done with our specific application requirement, with each layer having only two possible states, “on the server” or “on the client”, it is actually possible to extend the method to situations where each layer’s computation can be done in several possible different ways or states. This generalized approach is proven below, with “2 possible states” being a specific case for our algorithm.

Let’s consider a Directed Acyclic Graph (DAG) $G = (V, E)$. Let each vertex $v \in V$ have certain weight w_v , and let each edge $e(v_1, v_2) \in E$ from vertex v_1 to vertex v_2 have a weight $w_e(v_1, v_2)$. In this directed graph G , consider a source node v_s and a destination node v_d . The value/reward obtained by visiting any node v_i is given by r_i . Consider the dynamic

Table: Client(C)		Capacity →		
Layers →		$w = m$	$w = m + 1$	...
				...
	l_n	$\max(s2c, c2c)$	$\max(s2c, c2c)$	...
	l_{n+1}	$\max(s2c, c2c)$	$\max(s2c, c2c)$	...
	$\vdots$	...	...	...
Table: Server(S)		$w = m$	$w = m + 1$	...
				...
l_n		0	Run at server	Run at server

图 2. 算法 1 中描述的动态分配表的图示

在客户端上，当前层将在服务器上计算）。与每个配置相关的时间成本是相应计算的：例如， $c2c$ 仅具有 k^{th} 层中的计算成本，因为停留在同一设备中不会产生任何通信成本。

需要确保矩阵 C 和 S 的 j_{th} 列的成本只考虑可行的动作，因此导致这些成本超过 j 的情况只会有 ϵ 的值。例如， $s2c$ 需要有总成本值 $i_k + d_k$ (才能下载到客户端，然后在客户端中处理)，并且该值应小于或等于可用性值 j 。另外，第 8 行确保始终有足够的预算，以确保有足够的资源上传到服务器；这是通过在上传不可行时为 $c2s$ 分配一个非常小的值 ϵ 来完成的。该算法会自动忽略具有如此大负值的策略。最后，一旦计算出给定 j 和 k 的所有关联成本，那么 $C(k, j)$ 和 $S(k, j)$ 的值现在就可利用，并且这些值有助于分别在客户端或服务器设备上确定与在成本/时间预算 j 下拥有 k 层相关的成本。从左上角开始，首先从左到右填充每一行的表格，然后按照留置权 6-7 的建议移动到下面的行，直到填满整个表格。

图 2 直观地展示了为表赋值的过程。例如，让我们假设任务在服务器 S (的 l_{n-1} 处是最优/分配的，只是为了说明当最优分配在 S 处的 l_n 时会发生什么。然后，选项是将任务卸载到客户端 C 以在层 l_{n+1} 进行处理，成本为前面描述的 $s2c$ ，或者继续在服务器中运行。对于客户端 C ， l_{n+1} 层的最佳决策是在 S 获取数据和处理 l_n 的输出之间的更好选择，或者继续在客户端运行的策略，因此 $\max(s2c, c2c)$ 。

算法 1 的第二部分，第 19-39 行，用于使用现在填充的表 C 和 S 来辨别最佳策略 π ，用于将每一层分配给服务器或客户端。

我们首先将值 W 分配给变量 w ，该变量将跟踪已花费了多少时间资源。目标是跟踪已分配给已做出决策的层的整数化变量 w ($0 \leq w \leq W$)。我们开始在矩阵 C 和 S 上自下而上迭代，从最后一层的决策开始，到第一层结束。对于任何层 k ，是否将任务保留在客户端的决定由 $\pi[k]$ 中的条目决定。算法的这一部分中的第一步（第 21-24 行）是确定 k 层达到所选状态（到达客户端或服务器）的成本是多少，考虑到与处理层相关的不同成本，并在需要上传/下载时传输层输出。第 21 行假设为 $w > i_k$ ，其中 w 表示剩余时间资源， i_k 是在 i^{th} 层中客户端设备中处理数据的成本。 $c2c$ 的值表示在客户端设备上运行 k^{th} 层的决定用完剩余的预算，假设 k^{th} 层也在客户端上运行。同样， $c2s$ 表示当 k^{th} 层在客户端上运行而 $k+1^{th}$ 在服务器上运行时相同。 $s2c$ 和 $c2s$ 的计算方式类似。

下一步是根据 $k+1^{th}$ 层的值确定 k^{th} 层的分配，以及相关的处理和传输成本。如果根据决策变量 $\pi[k+1]$ 为 0（第 25 行）决定要在客户端中处理 $k+1^{th}$ 层，则下一步将比较成本以确定给定步骤是否会在服务器 ($\pi[k] = 1$) 或客户端 ($\pi[k] = 0$ 上进行最佳分配)。该决策是通过比较 $S(k, s2s)$ 和 $C(k, s2c)$ 处的值来做出的，以确定这两个分配中的哪一个将为未来层留下最多的资源。做出决定后，删除 k^{th} 层使用的资源非常重要。例如，在第 28 行中，此时已决定 $k+1^{th}$ 层将在服务器中处理，而 k^{th} 层将在客户端中运行。然后，新的权重由 $w - s2c$ 给出，其中 $s2c$ 是在客户端上运行 k^{th} 层的成本，由 i_k 给出，加上从客户端向服务器发送数据的成本 d_{ko} 。

算法 1 完成后，得到最优放置策略 π 。

C. Proof of Optimality

虽然问题的表述是根据我们的特定应用程序要求完成的，每层只有两种可能的状态，“在服务器上”或“在客户端”，但实际上可以将该方法扩展到每层的计算可以以几种可能的不同方式或状态完成的情况。下面证明了这种通用方法，其中“2 种可能的状态”是我们算法的具体情况。

让我们考虑一个有向无环图 (DAG) $G = (V, E)$ 。令每个顶点 $v \in V$ 具有一定的权重 w_v ，并令从顶点 v_1 到顶点 v_2 的每条边 $e(v_1, v_2) \in E$ 具有权重 $w_e(v_1, v_2)$ 。在此有向图 G 中，考虑源节点 v_s 和目标节点 v_d 。访问任意节点 v_i 获得的价值/奖励由 r_i 给出。考虑动态

programming approach, and consider a node v_i with all the nodes leading to v_i given by $v \in V_b(i)$.

In the dynamic programming approach, for each node, v_i , we assign a vector S_i of size W such that $S_i(w) \forall w \in W$ gives the best value for the assignment constraint of size w , which means that for a budget of w , the value stored will be the best possible one.

This implies that in the DAG, each of the possible neighbors $v_k \in V_b(i)$ is also assigned a vector S_k . So the value for $V_i(w)$ is given by:

$$V_i(w) = \max(r_i + V_k(w - w_e(v_k, v_i)) \forall v_k \in V_b(i)) \quad (3)$$

To show that this method always generates the optimal result, we proceed with proof by induction. So we begin with the first entry where $i = 1$. In this case, for any $w \in W$, the best result is always achieved with the possible inclusion of v_1 if possible, since no other nodes are considered.

Next, it should be shown that if the optimal solutions are provided for $\forall v_k \in V_b(i)$, then the optimal solution also exists for v_i following equation 3. Let this best value be obtained by reaching node v_i through node $v_{\hat{k}}$; then the maximum value is $r_i + V_{\hat{k}}(w - w_e(v_{\hat{k}}, v_i))$, where r_i is a constant for any w , and by assumption for $\forall V_k(w)$, $V_{\hat{k}}$ must be an optimal value.

And since v_d is an arbitrary node in the DAG, this must also be true for the destination node.

IV. DISCUSSION AND EVALUATION

In this section, we evaluate the results for the method discussed in section III. We start with subsection IV-A, where we use a sample case to demonstrate that intelligent splitting decisions can help reduce the server workload more efficiently, i.e., without causing a significant delay in end-to-end latency. In subsection IV-B, we demonstrate how efficient splitting could also be beneficial when the attention layers are approximated with low-rank or sparse representations (however, we do not consider these methods during the examinations in the subsequent sections, since our primary goal is to design a method that provides a performance guarantee). Then in subsection IV-C we discuss, by analyzing different types of transformer models, how the efficient splitting method we developed can help reduce server load while respecting the task completion deadline. We will demonstrate the efficiency of the method across different network and model features, including different sequence lengths of the input data, latency requirements, and bandwidth availability. Finally in subsection IV-D, we show that the developed method is capable of improving throughput for the servers providing the resources for different types LLM inference, under different SLAs and network conditions.

A. Examinations for Efficient Splitting

In a split learning or inference model, if each of the layers were similar in terms of computation requirements, or in terms of each layer's output size, then there would not be a need to invest resources towards optimal splitting policy, and there would be no need to split multiple times either. However, as we

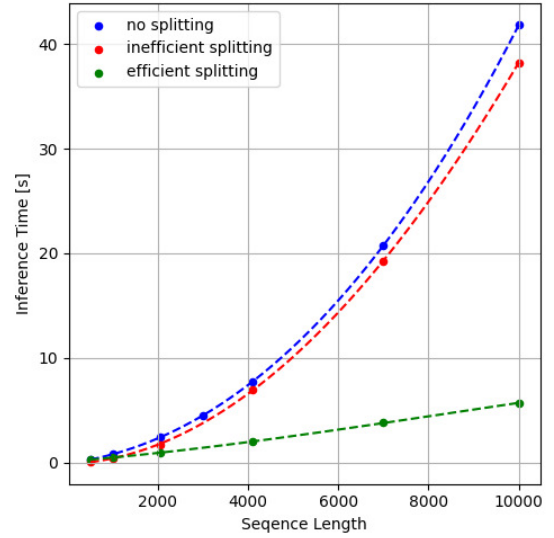


Fig. 3. Inference time under different split policies

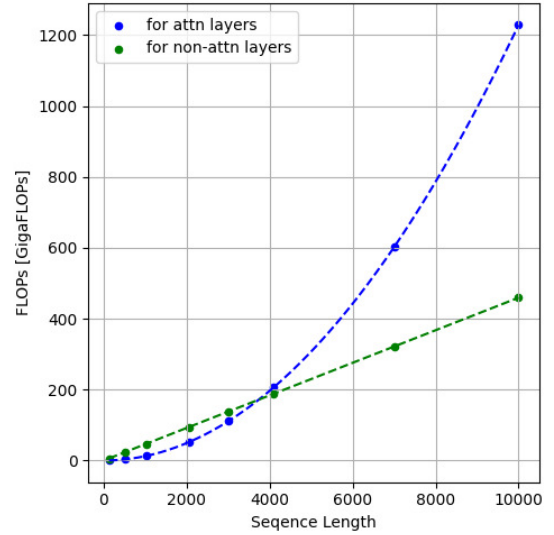


Fig. 4. FLOP used by different types of layers

have discussed earlier, the uneven nature of the steps involved in the inference of the LLMs, as well as the need for network efficiency during communication, means that an efficient splitting can help reduce the computation and communication workloads and lead to throughput improvements on the server side. To demonstrate the scope of efficient decision-making during split inference, we begin with a simple setup and some heuristic demonstrations.

We begin by discussing an experiment that shows the promise of efficient splitting for transformer-driven models by deploying a model with 12 attention layers among other components such as positional encoders and classifier (as used in BERT [2], or ALBERT [60] architecture). For this task, we set up a distributed learning environment with RTX 3090 as the server with GPU, and a resource-constrained 1 CPU core

编程方法，并考虑节点 v_i 以及由 $v \in V_b(i)$ 给出的通向 v_i 的所有节点。

在动态规划方法中，对于每个节点 v_i ，我们分配一个大小为 W 的向量 S_i ，使得 $S_i(w) \forall w \in W$ 为大小为 w 的分配约束提供最佳值，这意味着对于 w 的预算，存储的值将是可能的最佳值。

这意味着在 DAG 中，每个可能的邻居 $v_k \in V_b(i)$ 也被分配了一个向量 S_k 。因此 $V_i(w)$ 的值由下式给出：

$$V_i(w) = \max(r_i + V_k(w - w_e(v_k, v_i)) \forall v_k \in V_b(i)) \quad (3)$$

为了证明这种方法总是能产生最佳结果，我们通过归纳法进行证明。因此，我们从第一个条目开始，其中 $i = 1$ 。在这种情况下，对于任何 $w \in W$ ，如果可能的话，通过可能包含 v_1 始终可以获得最佳结果，因为没有考虑其他节点。

接下来，应该证明，如果为 $\forall v_k \in V_b(i)$ 提供了最优解，那么对于 v_i 也存在如下方程3的最优解。让这个最佳值通过节点 v_k 到达节点 v_i 来获得；则最大值为

$r_i + V_k(w - w_e(v_k, v_i))$ ，其中 r_i 是任何 w 的常数，并且假设 $\forall V_k(w)$ 、 V_k 一定是一个最优值。

由于 v_d 是 DAG 中的任意节点，因此对于目标节点也必须如此。

四. 讨论与评价

在本节中，我们评估第三节中讨论的方法的结果。我们从第 IV-A 小节开始，其中我们使用示例案例来证明智能拆分决策可以帮助更有效地减少服务器工作负载，即不会导致端到端延迟的显着延迟。在第 IV-B 小节中，我们演示了当注意力层用低秩或稀疏表示近似时，高效分割如何也是有益的（但是，在后续部分的检查中，我们不会考虑这些方法，因为我们的主要目标是设计一种提供性能保证的方法）。然后在第 IV-C 小节中，我们通过分析不同类型的变压器模型，讨论我们开发的高效分割方法如何在尊重任务完成期限的同时帮助减少服务器负载。我们将展示了该方法在不同网络和模型特征上的效率，包括不同的输入数据序列长度、延迟要求和带宽可用性。最后，在 IV-D 小节中，我们证明了所开发的方法能够在不同的 SLA 和网络条件下提高为不同类型的 LLM 推理提供资源的服务器的吞吐量。

A. Examinations for Efficient Splitting

在分割学习或推理模型中，如果每一层在计算要求或每层输出大小方面相似，那么就不需要为最优分割策略投入资源，也不需要多次分割。然而，正如我们

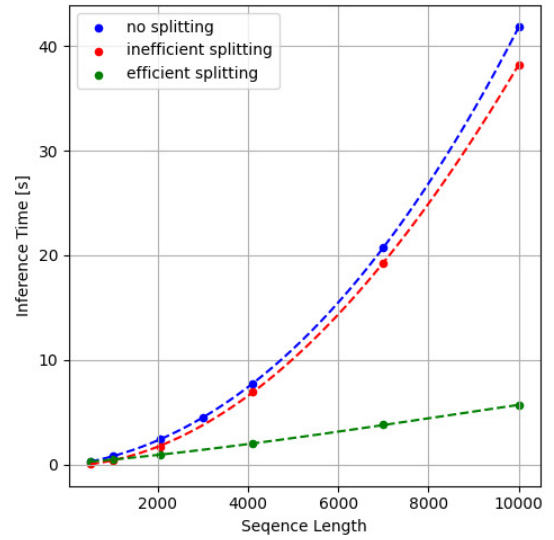


图 3. 不同分割策略下的推理时间

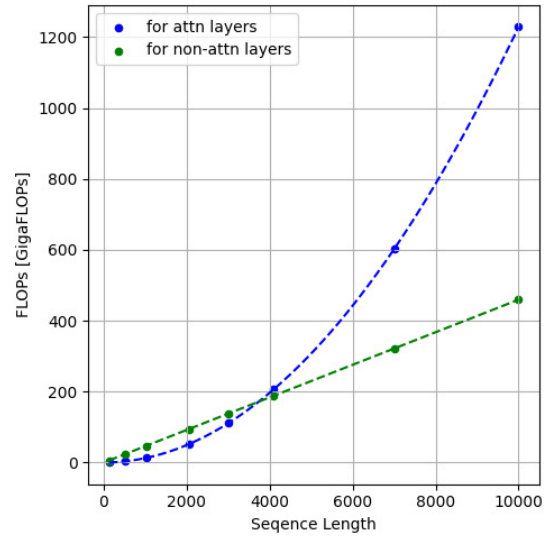


图 4. 不同类型层使用的 FLOP

正如前面所讨论的，LLM 的推理步骤的不均匀性，以及通信过程中对网络效率的需求，意味着有效的拆分可以帮助减少计算和通信工作量，并导致服务器端吞吐量的提高。为了演示吐出推理过程中高效决策的范围，我们从一个简单的设置和一些启发式演示开始。

我们首先讨论一个实验，该实验通过在位置编码器和分类器（如 BERT [2] 或 ALBERT [60] 架构中使用的）等其他组件中部署具有 12 个注意力层的模型，展示了 Transformer 驱动模型高效分割的前景。对于这个任务，我们建立了一个分布式学习环境，以 RTX 3090 作为带有 GPU 的服务器，以及资源受限的 1 个 CPU 核心

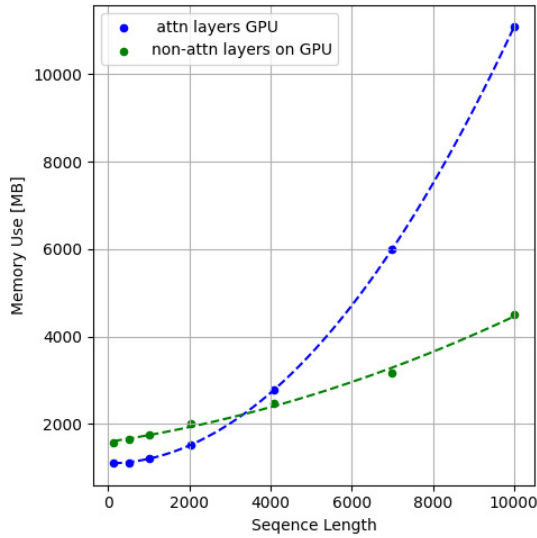


Fig. 5. Memory usage by different types of layers

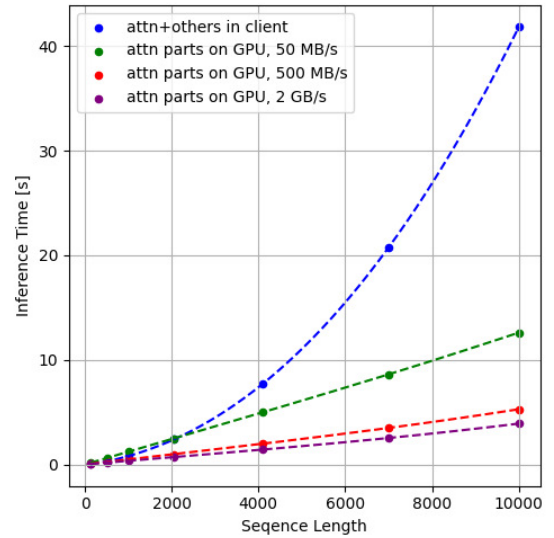


Fig. 6. Inference time under different split policies and bandwidths

(limited using taskset) as the client. In this split paradigm, the model is deployed with PyTorch, and TCP socket programming allows for communication between the server and the client. Then, we observe the inference across different input sequence lengths, s , with different splitting schemes, including a “no splitting” scenario where all computations are carried on the weak end device, a somewhat “efficient splitting” scenario where only the attention tasks are handled by the server/GPU, and “inefficient splitting” where all tasks except attention layers are handled by the server. Each results are obtained by running the inference 5 times and taking an average. It must be noted that the models aren’t trained for all the demonstrated sequence lengths, since in most cases larger sequence lengths lead to very high learning time, inference time, and resource costs (something we partially attempt to alleviate with this work). One of the objectives of this work is to develop a framework for larger input sequence lengths as well, which would allow for inference over larger inputs in future research and commercial endeavors.

The results of this experiment are shown in figure 3. “no splitting” in the figure represents a scenario where no splitting happens and everything is run on the client, while “inefficient” represents a very inefficient scenario where only the computationally intensive attention layers are run on the client. Finally, “efficient splitting” represents a scenario where only the attention layers are run on the server. As we observe in the figure, the quadratic nature of self-attention layers has a significantly adverse effect on the inference completion time, especially as the sequence length increases. As a result, it becomes much more preferable to run certain layers on a powerful server where a significant reduction of computation load can be achieved through powerful GPUs.

In Figure 4, we see the relative FLOP (Floating Point Operations) allocated to different types (attention vs. non-attention) of layers across different sequence lengths. These

values were analyzed for different types of attention, feed-forward, and other layers mathematically, but also verified later using fvcare [58] library. It can be seen that as the input data size grows, the number of FLOP allocated to these different types of layers behave differently; the other layers show a linear growth, while the attention layers have the FLOP values increasing quadratically. Let us begin by considering a situation where $s = 4000$ in figures 3 and 4. Here, while roughly similar computational cost is assigned to either the attention or the non-attention layers (figure 4), the inference completion times are very different for two the different splitting schemes. For instance, the latency requirements can be 4 times more relaxed / higher in efficient splitting vs. inefficient splitting while a similar resource-saving is achieved. This effect can be observed across different optimization objectives. In figure 5, we observe the same resource-saving potential for GPU memory allocation for inference tasks, where attention layers occupy slightly more GPU memory (roughly 1.2x) at $s = 4000$, but the inference time is 4 times less as before. Hence, it becomes obvious that relatively more or less efficient policies exist for different optimization goals, further validating the need for formulation and algorithm in section III. In this section, we will rely on FLOP count for analysis, but the formulation and algorithm are completely agnostic to which computation resource is selected for minimization.

To ascertain the efficiency of the method, one of the earlier tests conducted was to see how the method could work for different bandwidth availability. In figure 6, we can observe that the benefit of efficient splitting discussed above could be observed across different bandwidths, and the improvement is more pronounced as we increase the available bandwidth. The improvement is seen across the different input sequence lengths, as the green, red, and purple lines are significantly below the blue curve, especially as the sequence length increases. Another point to ascertain would be the assumption

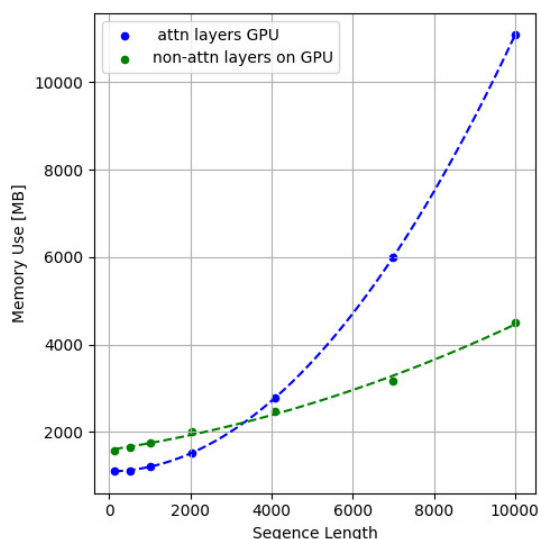


图 5. 不同类型层的内存使用情况

（限制使用任务集）作为客户端。在这种分离范例中，模型是用 PyTorch 部署的，TCP 套接字编程允许服务器和客户端之间的通信。然后，我们观察不同输入序列长度 s 的推理，使用不同的分割方案，包括所有计算都在弱端设备上进行的“无分割”场景，仅注意力任务由服务器/GPU 处理的有点“高效分割”场景，以及除注意力层之外的所有任务都由服务器处理的“低效分割”场景。每个结果都是通过运行推理 5 次并取平均值获得的。必须注意的是，模型并未针对所有演示的序列长度进行训练，因为在大多数情况下，较大的序列长度会导致非常高的学习时间、推理时间和资源成本（我们试图通过这项工作部分缓解这一问题）。这项工作的目标之一是开发一个适用于更大输入序列长度的框架，这将允许在未来的研究和商业活动中对更大的输入进行推断。

这个实验的结果如图3所示。图中的“no splitting”代表不发生任何分裂并且一切都在客户端运行的场景，而“inefficient”则代表只在客户端运行计算密集型注意力层的非常低效的场景。最后，“高效分割”代表了仅注意力层在服务器上运行的场景。正如我们在图中观察到的，自注意力层的二次性质对推理完成时间有显著的不利影响，特别是当序列长度增加时。因此，在强大的服务器上运行某些层变得更加可取，通过强大的 GPU 可以显著减少计算负载。

在图 4 中，我们看到分配给不同序列长度的不同类型（注意力与非注意力）层的相对 FLOP（浮点运算）。这些

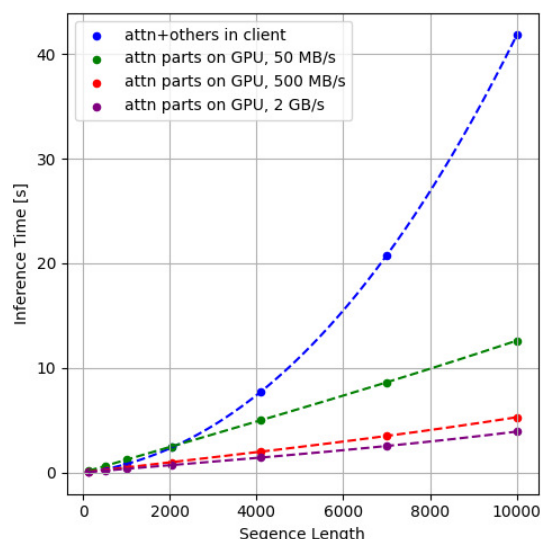


图 6. 不同分割策略和带宽下的推理时间

对不同类型的注意力、前馈和其他层的值进行了数学分析，但后来也使用 fvcare [58] 库进行了验证。可以看出，随着输入数据大小的增长，分配给这些不同类型层的 FLOP 数量表现不同；其他层呈现线性增长，而注意力层的 FLOP 值呈二次方增长。让我们首先考虑图 3 和图 4 中 $s = 4000$ 的情况。这里，虽然分配给注意力层或非注意力层的计算成本大致相似（图 4），但对于两种不同的分割方案，推理完成时间却截然不同。例如，与低效分割相比，高效分割的延迟要求可以放宽/更高 4 倍，同时实现类似的资源节省。这种效果可以在不同的优化目标中观察到。在图 5 中，我们观察到推理任务的 GPU 内存分配具有相同的资源节省潜力，其中注意力层在 $s = 4000$ 时占用略多的 GPU 内存（大约 1.2 倍），但推理时间比以前减少了 4 倍。因此，很明显，对于不同的优化目标，存在相对或多或少有效的策略，进一步验证了第三节中公式和算法的需要。在本节中，我们将依靠 FLOP 计数进行分析，但公式和算法完全不知道选择哪个计算资源进行最小化。

为了确定该方法的效率，早期进行的测试之一是了解该方法如何适用于不同的带宽可用性。在图 6 中，我们可以观察到，上面讨论的高效拆分的好处可以在不同的带宽上观察到，并且随着可用带宽的增加，这种改进更加明显。在不同的输入序列长度上都可以看到这种改进，因为绿色、红色和紫色线明显低于蓝色曲线，特别是随着序列长度的增加。另一点需要确定的是假设

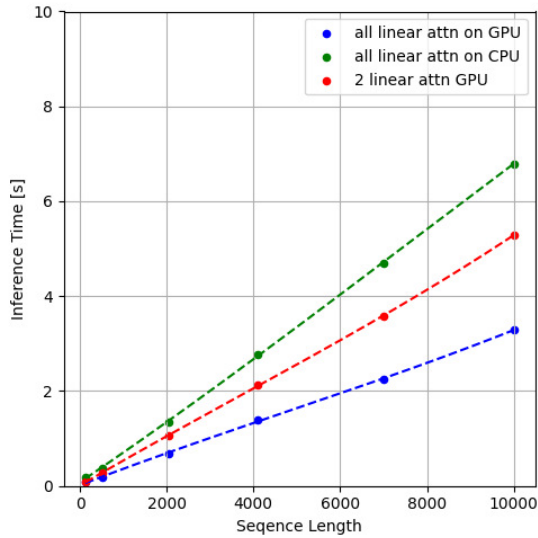


Fig. 7. Inference time for different splitting policies when linear approximation of attention matrix is used.

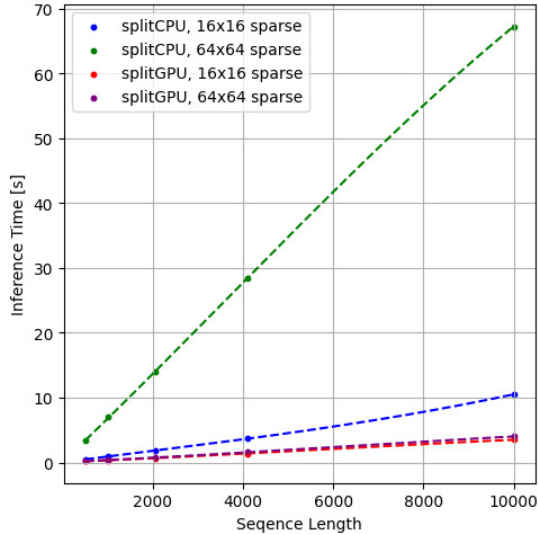


Fig. 8. Inference time for different splitting policies when sparse approximation of attention matrix is used.

that server implementation was significantly faster than client implementation. Our implementation showed that running the entire model with sequence length 4096 would take 7.727 seconds in the client, while it would only take 0.0979 seconds in the server, verifying the assumption that completion time is much faster on the server side.

B. Sparse and Low Rank Approximation

While the goal of this paper is to demonstrate the benefits of collaborative inference without a loss in optimal performance, the formulation could be extended to situations where approximate models that trade off accuracy for better inference time and lower computation cost are deployed. This includes models with sparse or low-rank representations of the self-

attention matrix. As a proof-of-concept, we demonstrate how efficient policies could be formulated in such scenarios. In figure 7, we refer to a solution, i.e., [12], where a linear combination of low-rank matrices helps approximate the full attention layer. As we see, for different sequence lengths, depending on the latency requirements, all, none, or 2/3 of the linear layers could be run on the GPU. Similarly, in figure 8, we can see that different sparse approximations (with different smaller matrix sizes [13]) can be used to achieve different collaborative inference results. Here, splitCPU refers to transmitting attention layers to another machine without GPU, while splitGPU refers to transmitting attention layers to another device with GPU; the size, i.e., 16×16 , is the size of the smaller matrices used to approximate full attention matrix.

C. Effectiveness of the Method

In this section, our primary objective is to demonstrate the efficiency of our method, based around algorithm 1, for efficiently optimizing the load reduction objective as formulated in equation 2, while ensuring that the end-to-end inference latency constraint is always satisfied. We begin with an experimental setup as described in the preceding section IV-C with a couple of changes. First, we are not limited to one model but experiment across different models, including multiple LLMs and an image-recognition transformer. We simulate the inter-device communication this time, where instead of using a socket programming approach, the communication delay and transmission delay are simulated, which greatly saves time in running inference across different bandwidths, models, and more. We opt for this approach since the efficacy of the approach is already established in the earlier subsection.

We compare our algorithm against the 'Greedy' approach, where the computation is greedily assigned to the client device for the first layers, so long as the latency constraint allows it. This is based on the split decision-making as described in [29]. Solutions that stem from such splitting, including [25], [53], [61], are complementary to our method. Greedy splitting is proposed as the offline solution in [62] for a non-server-client non-LLM-specific paradigm.

Before running inference on the models, the proof of optimality presented in section III-C was complemented with numerical verification. Random number generators were used to demonstrate that the dynamic programming approach is optimal, and the same random number generator was used to show that the algorithm performs better than the 'Greedy' approach. This was done by running the numerical calculations multiple times. The greedy approach entails computing as many layers as possible on the client side, but the selection mechanism is greedy, in that the layers that come earlier are always selected until the budget runs out. Then the remaining layers have to be processed on the server side.

Before trying different models, we begin with a model as designed in [8], which is a transformer with 6 encoder and 6 decoder layers leading to 18 self-attention layers, plus the adjacent feed-forward NNs and other layers, alongside the positional encoding layers, classifier and more. As before, we

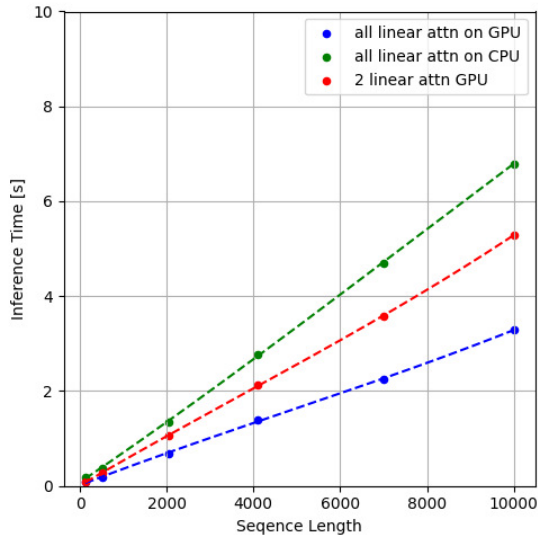


图 7. 使用注意力矩阵的线性近似时不同分割策略的推理时间。

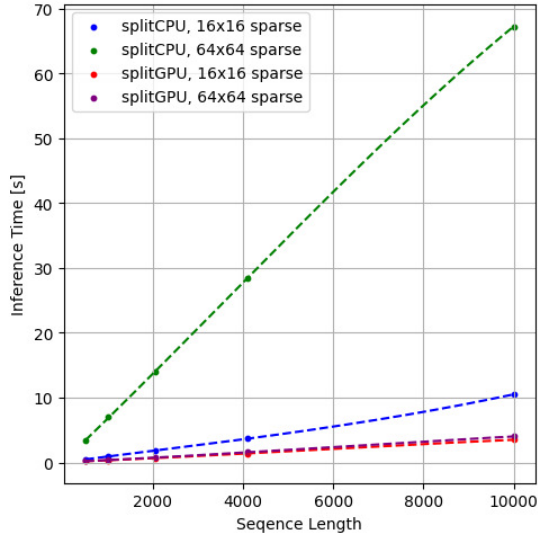


图 8. 使用注意力矩阵的稀疏近似时不同分割策略的推理时间。

服务器实现明显快于客户端实现。我们的实现表明，在客户端中运行序列长度为 4096 的整个模型将花费 7.727 秒，而在服务器中只需要 0.0979 秒，验证了服务器端完成时间要快得多的假设。

B. Sparse and Low Rank Approximation

虽然本文的目标是在不损失最佳性能的情况下展示协作推理的好处，但该公式可以扩展到部署近似模型的情况，该近似模型需要权衡准确性以换取更好的推理时间和更低的计算成本。这包括具有稀疏或低秩表示的模型

注意力矩阵。作为概念验证，我们展示了如何在这种情况下制定有效的政策。在图 7 中，我们参考了一个解决方案，即 [12]，其中低秩矩阵的线性组合有助于近似完整的注意力层。正如我们所看到的，对于不同的序列长度，根据延迟要求，所有、没有或 2/3 个线性层都可以在 GPU 上运行。同样，在图 8 中，我们可以看到不同的稀疏近似（具有不同的较小矩阵大小 [13]）可用于实现不同的协作推理结果。这里，splitCPU 是指将注意力层传输到另一台没有 GPU 的机器上，而 splitGPU 是指将注意力层传输到另一台有 GPU 的设备上；大小，即 16×16 ，是用于近似全注意力矩阵的较小矩阵的大小。

C. Effectiveness of the Method

在本节中，我们的主要目标是证明我们的方法的效率，基于算法 1，有效优化等式 2 中制定的负载减少目标，同时确保始终满足端到端推理延迟约束。我们从前面第 IV-C 节中描述的实验设置开始，并进行了一些更改。首先，我们不限于一种模型，而是对不同模型进行实验，包括多个 LLM 和图像识别转换器。这次我们模拟设备间通信，没有使用 socket 编程方式，而是模拟通信延迟和传输延迟，这大大节省了跨不同带宽、模型等运行推理的时间。我们选择这种方法，因为该方法的功效已经在前面的小节中确立。

我们将我们的算法与“贪婪”方法进行比较，在“贪婪”方法中，只要延迟约束允许，计算就会贪婪地分配给第一层的客户端设备。这是基于 [29] 中描述的分裂决策。源于这种分裂的解决方案，包括 [25]、[53]、[61]，与我们的方法是互补的。贪婪分割被提出作为 [62] 中非服务器-客户端非 LLM 特定范例的离线解决方案。

在对模型进行推理之前，第 III-C 节中提出的最优性证明通过数值验证进行了补充。使用随机数生成器来证明动态规划方法是最优的，并且使用相同的随机数生成器来证明该算法比“贪婪”方法表现得更好。这是通过多次运行数值计算来完成的。贪婪方法需要在客户端计算尽可能多的层，但选择机制是贪婪的，因为总是选择较早出现的层，直到预算用完。然后剩下的层必须在服务器端处理。

在尝试不同的模型之前，我们从 [8] 中设计的模型开始，它是一个具有 6 个编码器和 6 个解码器层的转换器，导致 18 个自注意力层，加上相邻的前馈神经网络和其他层，以及位置编码层、分类器等。和以前一样，我们

run the inference 5 times and take the average to get inference data. We also select different bandwidths and assign latency so that the latencies roughly fall in the range where roughly all, to almost none, of the data can be offloaded to the client devices. Each subsequent latency is half of the larger one, and this way the data are not unfairly selected. Similarly, different transmission rates are selected as well, and a communication delay of 10 ms is added. It must be noted that such splitting paradigms are suitable under different environments including the fog paradigm with fairly decent communication resources. In the communication infrastructures of the past such as 4G cellular networks, such learning paradigms would not be as suitable since significant network resources are consumed.

In figure 9, we observe the amount of computation offloaded to the server; While these values differ across different latencies, sequence lengths, and different model settings, the average percent of resources offloaded to the server was 28.9 percent. With this ratio of inference tasks offloaded to the server, the improvement of our method over the greedy method was found to be 14.6 percent. In figure 10, we can see the improved performance over the greedy approach for most of the values across the sequence lengths and the latencies. These values, across latencies and sequence lengths, were averaged over different bandwidth availability. As is intuitive, we can observe in figures 9 and 10 that as the latency becomes high enough for most of the tasks to be offloaded to the client, the benefit over the greedy approach diminishes; the benefit is more pronounced for stricter latencies.

In figure 11, we see the average tasks offloaded to the server across different bandwidths, and as is intuitive, at higher bandwidths, it becomes easier to offload more tasks to the server, which is seen with the growth in the amount of resources that can be offloaded with the increasing bandwidth availability. In figure 12, we see the improvement of our method over the greedy approach, in reducing workload at the server, when compared across different transmission rates.

Next, we repeated the experiment for two other layers, i.e., BERT (12 encoder layers, so 12 attention layers) and GPT-2-like model (24 attention layers). In each of the cases, we allowed for roughly a bit under 1/3rd of the resources to be offloaded to the clients on average (27.8 percent for BERT, and 29.2 percent for GPT-2-like model). We mention “GPT-2-like” since the exact parameter space for GPT-2 isn’t fully public knowledge. For the case of BERT, the improvement over the greedy approach at efficiently offloading to the client was found to be 5.5 percent, while it was found to be 12.5 percent for the GPT-2-like model. This showed us that the improvement across different environments and model configurations, which we saw earlier, was also observed across different models. While it was not always the case, the fact that we generally saw improved performance for larger sequence lengths, as well as larger models, suggests that models with larger computational requirement/parameter size might see a bigger improvement.

Until this point, and the core objective of the paper, focuses on the language models, and the behavior of the language

models across different environments or model designs. While this analysis is not considered in other sections, we also attempted to understand the efficiency of our method on a transformer that specialized in visual recognition [7]. While we start with the default Imagenet [63] images of size 224x224, we scale it, just for inference tests, up to 4 times, to study the effect of input size on the splitting policy. We recognize that unlike the language models, where a longer input size is highly desirable and different efforts have been made towards achieving this, a larger input size isn’t as sought after in the domain of visual deep learning methods. So we do not use these results during the throughput consideration (section IV-D). Unlike language models, visual transformers are significantly deeper and the NN has a much more varying structure. Here, we found that with 44.3 percent network resource saved, the improvement over the greedy approach was 55.4 percent. Since the input sizes fluctuate a lot layer-after-layer in such visual transformers, and since the greedy algorithm needs to reserve certain time resources for uploading in the worst-case situation where the time deadline may come to an end while processing is still in the client device and output of the layer is large, the performance for the greedy method was worse for the visual transformer as opposed to the language models. Since our method guarantees optimality under the given relaxations, such situations are not of concern to our algorithm.

D. Throughput Improvement

In the earlier section, IV-C, we demonstrated that our method can lead to an efficient split inference such that the server load is minimized given the latency requirement. In this section, we will demonstrate how these reductions can lead to efficient throughput improvements during the dynamic scheduling of model inference requests. In this section, we will use the data collected across different models, bandwidth requirements, and latencies to simulate a random arrival process with an inter-arrival rate β . The arriving traffic is served by a server with a computational capacity described by Ω . The tasks that do not have enough resources provided upon arrival will stay in a queue that has a “first-in-first-out” principle, and the resource availability is frequently checked to see if the next request can be executed at the time. The running time of the tasks is based on the task completion deadline and the frequency of executions (some tasks may be asked to be executed multiple times, i.e., up to 10 times). Since this demonstration relies on high arrival rates and provisioning of the services to a large number of requests, we rely on simulations to demonstrate the improvements achieved from our method.

In figure 13, we observe the throughput when $\beta = 57/1000$. Capacity Ω here is described as being able to serve 500 requests on average (pre-calculated) at a given time, and the total number of requests served is 14,949. Here, we observed that the maximum wait time for our method was 1.36 s, while it was 3.13 s for the greedy method, and 110.62 s for the no-split method. The average wait times were 0.0061 s for our

运行推理5次，取平均值得到推理数据。我们还选择不同的带宽并分配延迟，以便延迟大致落在可以将所有数据（甚至几乎没有数据）卸载到客户端设备的范围内。随后的每个延迟都是较大延迟的一半，这样数据就不会被不公平地选择。同样，也选择不同的传输速率，并增加10ms的通信延迟。必须指出的是，这种分裂范式适用于不同的环境，包括通信资源相当不错的雾范式。在过去的通信基础设施（例如 4G 蜂窝网络）中，这种学习范例不太合适，因为会消耗大量网络资源。

在图 9 中，我们观察到卸载到服务器的计算量；虽然这些值因不同的延迟、序列长度和不同的模型设置而异，但卸载到服务器的资源的平均百分比为 28.9%。通过将推理任务卸载到服务器的比例，我们的方法相对于贪婪方法的改进为 14.6%。在图 10 中，我们可以看到对于序列长度和延迟上的大多数值而言，贪婪方法的性能有所提高。这些跨延迟和序列长度的值是在不同的可用带宽上取平均值的。直观地说，我们可以在图 9 和图 10 中观察到，随着延迟变得足够高以将大多数任务卸载到客户端，相对于贪婪方法的好处就会减少；对于更严格的延迟，好处更加明显。

在图 11 中，我们看到跨不同带宽卸载到服务器的平均任务，直观地说，在更高的带宽下，将更多任务卸载到服务器变得更容易，随着可用带宽的增加，可卸载的资源量也随之增加。在图 12 中，与不同传输速率进行比较时，我们看到我们的方法相对于贪婪方法的改进，减少了服务器的工作负载。

接下来，我们对另外两个层重复了实验，即 BERT（12 个编码器层，因此 12 个注意力层）和类似 GPT-2 的模型（24 个注意力层）。在每种情况下，我们平均允许将大约不到 1/3 的资源卸载给客户端（BERT 为 27.8%，类似 GPT-2 模型为 29.2%）。我们提到“类似 GPT-2”是因为 GPT-2 的确切参数空间并不完全为公众所知。对于 BERT 的情况，发现有效卸载到客户端的贪婪方法的改进为 5.5%，而对于类似 GPT-2 的模型，改进为 12.5%。这向我们表明，我们之前看到的不同环境和模型配置之间的改进也在不同模型中观察到。虽然情况并非总是如此，但我们通常会看到较大序列长度以及较大模型的性能得到改善，这一事实表明具有较大计算要求/参数大小的模型可能会看到更大的改进。

到目前为止，本文的核心目标是关注语言模型和语言的行为

跨不同环境或模型设计的模型。虽然其他部分没有考虑这种分析，但我们还尝试了解我们的方法在专门用于视觉识别的变压器上的效率[7]。虽然我们从大小为 224x224 的默认 Imagenet [63] 图像开始，但为了推理测试，我们将其缩放至 4 倍，以研究输入大小对分割策略的影响。我们认识到，与语言模型不同，在语言模型中，非常需要更长的输入大小，并且为实现这一目标付出了不同的努力，而在视觉深度学习方法领域中，更大的输入大小并不受欢迎。因此，我们在考虑吞吐量时不使用这些结果（第 IV-D 节）。与语言模型不同，视觉转换器明显更深，神经网络的结构也更加多样化。在这里，我们发现节省了 44.3% 的网络资源，比贪婪方法的改进为 55.4%。由于在这种视觉变换器中输入大小逐层波动很大，并且由于贪婪算法需要在最坏的情况下保留一定的时间资源用于上传，在最坏的情况下，当处理仍在客户端设备中并且该层的输出很大时，时间期限可能会结束，因此对于视觉变换器来说，贪婪方法的性能比语言模型更差。由于我们的方法保证了给定松弛下的最优性，因此我们的算法不关心这种情况。

D. Throughput Improvement

在前面的 IV-C 部分中，我们证明了我们的方法可以实现有效的分割推理，从而在考虑到延迟要求的情况下最大限度地减少服务器负载。在本节中，我们将演示这些减少如何在模型推理请求的动态调度期间实现有效的吞吐量提高。在本节中，我们将使用跨不同模型、带宽要求和延迟收集的数据来模拟具有到达间隔率 β 的随机到达过程。到达的流量由计算能力为 Ω 的服务器提供服务。到达时没有提供足够资源的任务将停留在具有“先进先出”原则的队列中，并且经常检查资源可用性以查看当时是否可以执行下一个请求。任务的运行时间取决于任务完成期限和执行频率（有些任务可能会要求执行多次，即最多10次）。由于此演示依赖于高到达率和为大量请求提供服务，因此我们依靠模拟来演示我们的方法所实现的改进。

在图 13 中，我们观察到 $\beta = 57/1000$ 时的吞吐量。这里的容量 Ω 被描述为在给定时间平均能够服务 500 个请求（预先计算），并且服务的请求总数是 14,949。在这里，我们观察到我们的方法的最大等待时间为 1.36 秒，而贪婪方法的最大等待时间为 3.13 秒，无分割方法的最大等待时间为 110.62 秒。我们的平均等待时间为 0.0061 秒

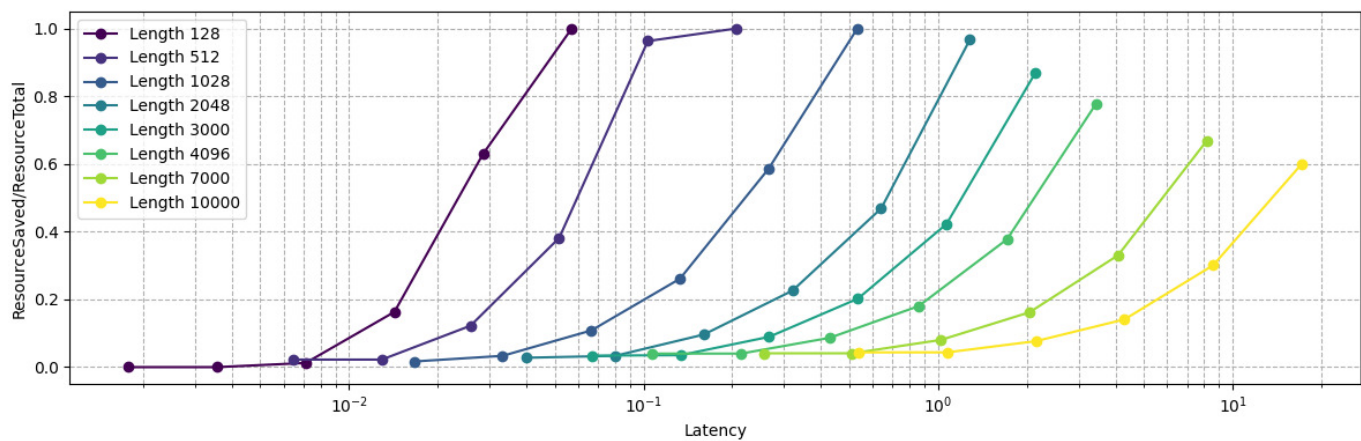


Fig. 9. Resource usage across different latency requirements

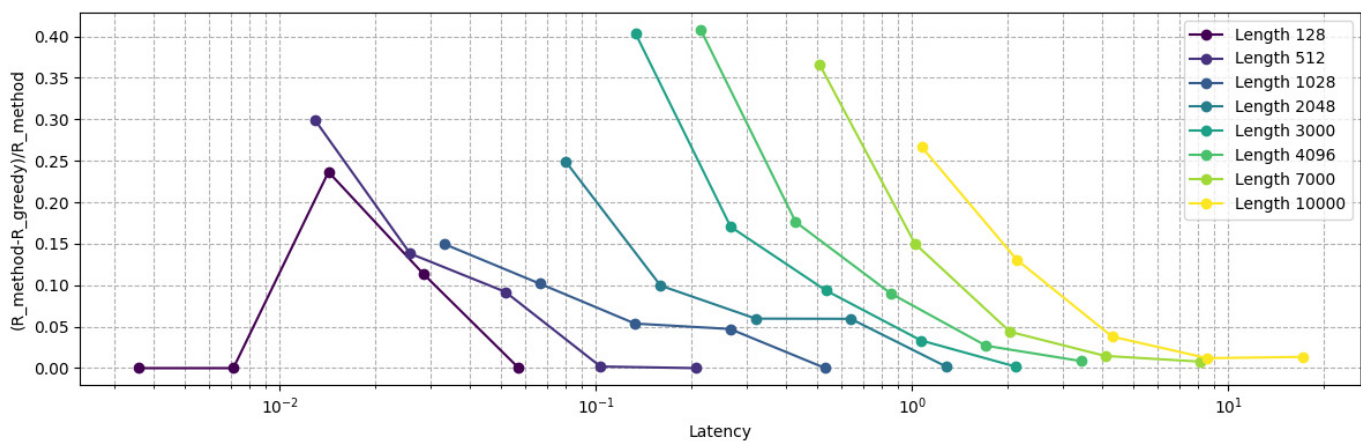


Fig. 10. Improvement over greedy method across different latency requirements

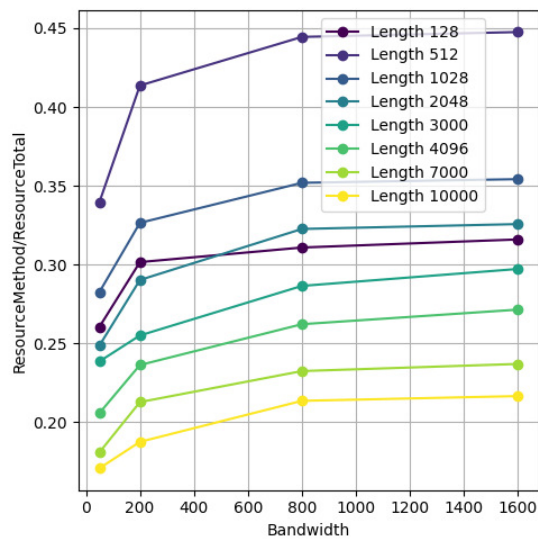


Fig. 11. Resource usage across different bandwidth availability

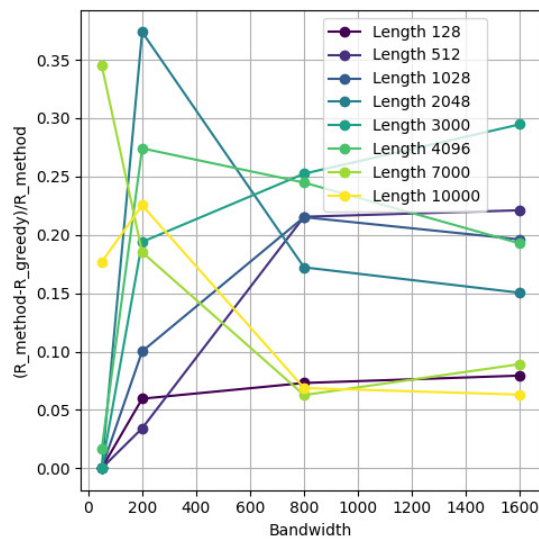


Fig. 12. Improvement over greedy method across different bandwidth availability

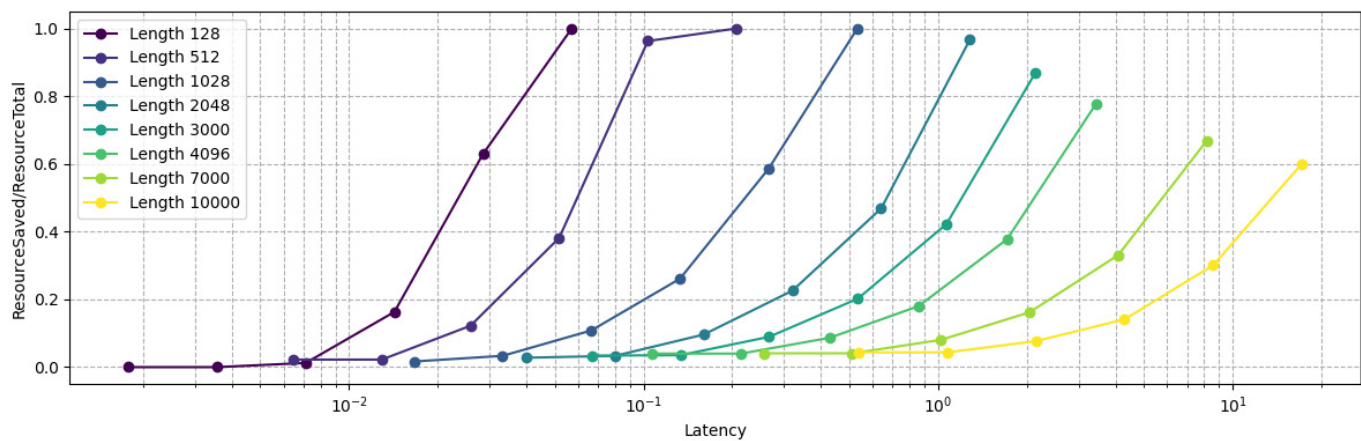


图 9. 不同延迟要求下的资源使用情况

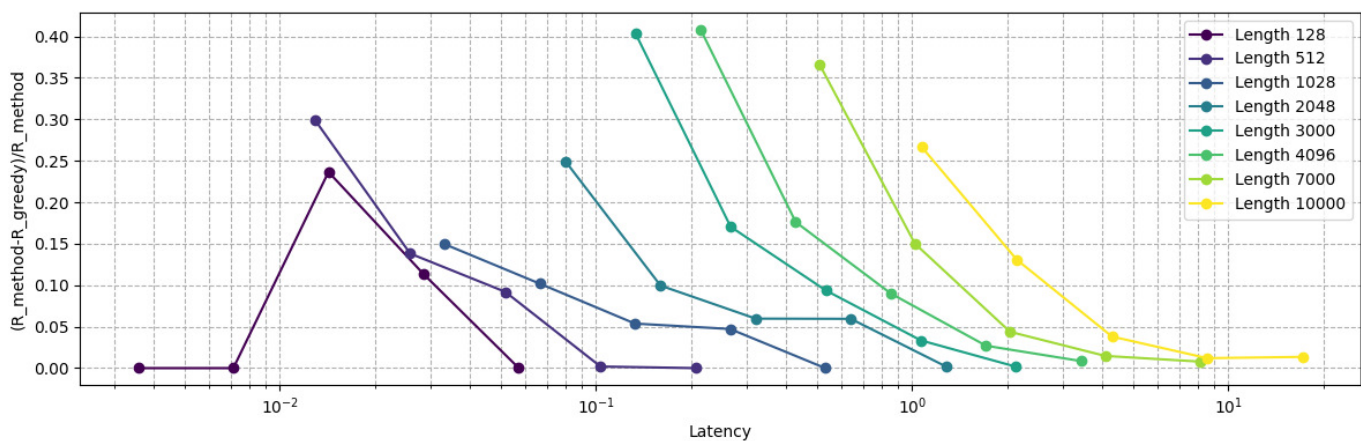


图 10. 针对不同延迟要求的贪婪方法的改进

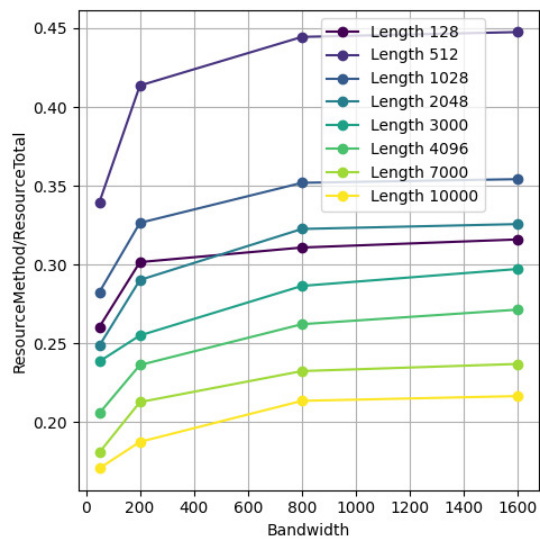


图 11. 不同带宽可用性下的资源使用情况

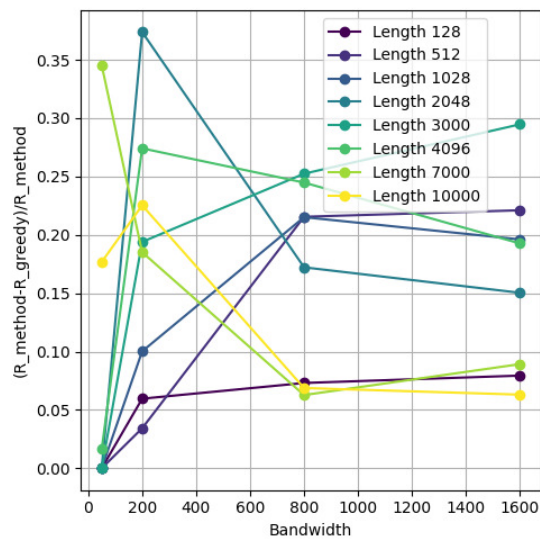


图 12. 在不同带宽可用性下对贪婪方法的改进

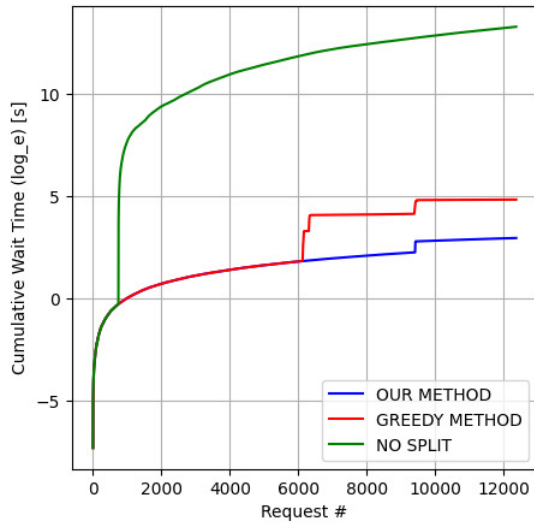


Fig. 13. cumulative wait time across different methods, $\beta = 57/1000$

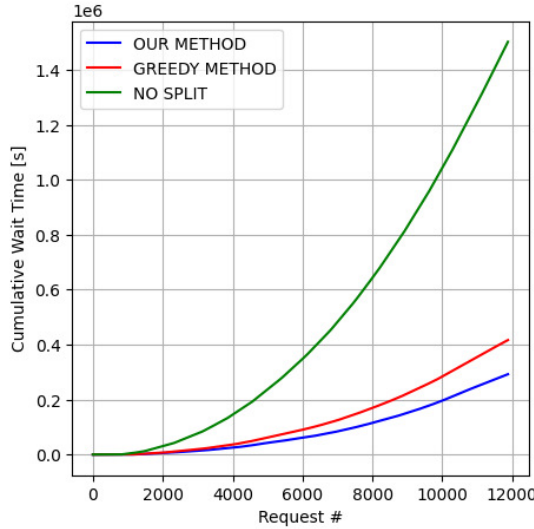


Fig. 14. cumulative wait time across different methods, $\beta = 45/1000$

method, 0.0682 s for the greedy method, and 47.507 s for the no-split method. This showed a significant improvement when our method is employed. For a different value $\beta = 45/1000$, we observe that our method has a higher cumulative queue wait time than before, but it is significantly lower than no-split or greedy approaches, as shown in figure 14. Here, as the arrival rate for the requests was increased and all methods faced a queue, the maximum delay grew to 59.2 s for our method, 87.2 s for the greedy method, and 270.5 s for the no split approach. This is a kind of delay that would violate service agreements in many cases, but it can be observed that our method still has a significantly better performance. On the other direction, if the rate was increased to $\beta = 60/1000$ for instance, the average wait time would be negligible for our method or greedy approach, while it would be 73.67 s for no

split approach.

The goal of this section was to show how efficient splitting policies can help with the improvement of throughput at the servers providing such LLM-based services. Needless to say, the server capacity should be designed to handle expected traffic in a data-centric way, but with our method, it can be observed that there is a better throughput performance across different workloads for a given server capacity. Such designs should be data-driven and well-planned, but a method that reduces the server load at the individual request level, such as ours, is bound to improve the performance in cumulative deployment scenarios.

V. CONCLUSION

As the LLMs have proven to be an extremely pervasive technology, new methods are needed to tackle the high costs of computation and resource congestion that such a growth is likely to pose. Recognizing that this issue will become progressively worse in the near future, we developed a collaborative inference scheme that exploits the nature of transformer models and provides an efficient splitting algorithm for the LLMs with different input sequence lengths, model types, network settings, and other requirements. We show that our method outperforms a greedy approach by 19 percent on average, towards decreasing computation costs at the server by roughly $1/3^{rd}$. We also show that this improvement in turn increases throughput at the server.

REFERENCES

- [1] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever *et al.*, “Improving language understanding by generative pre-training,” 2018.
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018.
- [3] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androuloupoulos, “Legal-bert: The muppets straight out of law school,” *arXiv preprint arXiv:2010.02559*, 2020.
- [4] J. Lee, W. Yoon, S. Kim, D. Kim, S. Kim, C. H. So, and J. Kang, “Biobert: a pre-trained biomedical language representation model for biomedical text mining,” *Bioinformatics*, vol. 36, no. 4, pp. 1234–1240, 2020.
- [5] D. Support, “Next-generation language model for deepl translator,” <https://support.deepl.com/hc/en-us/articles/14241705319580-Next-generation-language-model-for-DeepL-Translator>, 2024, accessed: 2024-05-25.
- [6] L. Schneider, A. Baevski, R. Collobert, and M. Auli, “wav2vec: Unsupervised pre-training for speech recognition,” *arXiv preprint arXiv:1904.05862*, 2019.
- [7] J. Guo, K. Han, H. Wu, C. Xu, Y. Tang, C. Xu, and Y. Wang, “Cmt: Convolutional neural networks meet vision transformers,” 2021.
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, E. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [9] L. De Angelis, F. Baglivo, G. Arzilli, G. P. Privitera, P. Ferragina, A. E. Tozzi, and C. Rizzo, “Chatgpt and the rise of large language models: the new ai-driven infodemic threat in public health,” *Frontiers in Public Health*, vol. 11, p. 1166120, 2023.
- [10] C. Huyen, “Multimodal models,” <https://short.url/multimodal>, 2023, accessed: 2024-05-25.
- [11] I. Beltagy, M. E. Peters, and A. Cohan, “Longformer: The long-document transformer,” *arXiv preprint arXiv:2004.05150*, 2020.
- [12] B. Chen, T. Dao, E. Winsor, Z. Song, A. Rudra, and C. Ré, “Scatterbrain: Unifying sparse and low-rank attention,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 17 413–17 426, 2021.

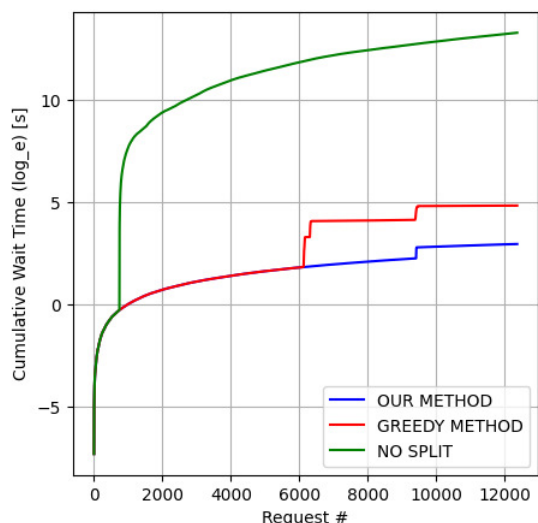


图 13. 不同方法的累积等待时间, $\beta = 57/1000$

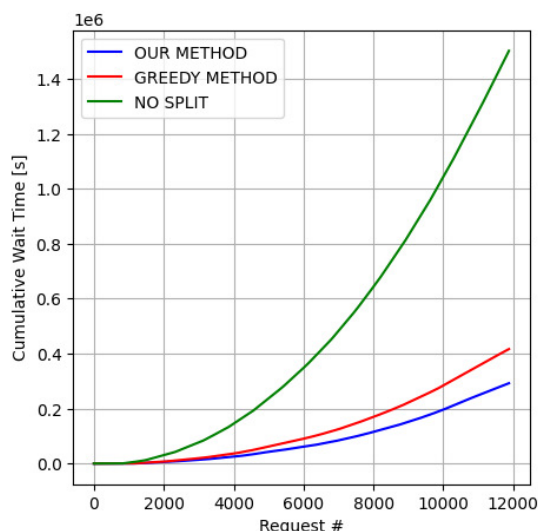


图 14. 不同方法的累积等待时间, $\beta = 45/1000$

方法, 贪婪方法为 0.0682 秒, 无分割方法为 47.507 秒。当采用我们的方法时, 这显示出显著的改进。对于不同的值 $\beta = 45/1000$, 我们观察到我们的方法比以前具有更高的累积队列等待时间, 但明显低于不分割或贪婪方法, 如图 14 所示。这里, 随着请求到达率的增加并且所有方法都面临队列, 我们的方法的最大延迟增长到 59.2 秒, 贪婪方法为 87.2 秒, 不分割为 270.5 秒方法。这是一种在许多情况下会违反服务协议的延迟, 但可以观察到我们的方法仍然具有明显更好的性能。另一方面, 如果速率增加到 $\beta = 60/1000$, 则对于我们的方法或贪婪方法, 平均等待时间可以忽略不计, 而对于没有方法, 平均等待时间将为 73.67 秒。

分裂的方法。

本节的目标是展示有效的拆分策略如何帮助提高提供此类基于 LLM 的服务的服务器的吞吐量。不用说, 服务器容量的设计应以以数据为中心的方式处理预期流量, 但通过我们的方法, 可以观察到, 对于给定的服务器容量, 跨不同工作负载有更好的吞吐量性能。这样的设计应该是数据驱动且精心规划的, 但是像我们这样在单个请求级别减少服务器负载的方法必然会提高累积部署场景中的性能。

五、结论

由于大语言模型已被证明是一种极其普遍的技术, 因此需要新的方法来解决这种增长可能带来的高计算成本和资源拥堵问题。认识到这个问题在不久的将来将变得越来越严重, 我们开发了一种协作推理方案, 该方案利用变压器模型的性质, 并为具有不同输入序列长度、模型类型、网络设置和其他要求的 LLM 提供有效的分割算法。我们表明, 我们的方法平均比贪婪方法高出 19%, 从而将服务器的计算成本降低了大约 $1/3^{rd}$ 。我们还表明, 这种改进反过来又提高了服务器的吞吐量。

参考

- [1] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever *et al.*, “通过生成预训练提高语言理解”, 2018 年。[2] J. Devlin, M.-W. Chang, K. Lee 和 K. Toutanova, “Bert: 用于语言理解的深度双向转换器的预训练”, *arXiv preprint arXiv:1810.04805*, 2018 年。[3] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras 和 I. Androuloupoulos, “Legal-bert: 法学院毕业的木偶”, *arXiv preprint arXiv:2010.02559*, 2020。[4] J. Lee, W. Yoon, S. Kim, D. Kim, S. Kim, C. H. So 和 J. Kang, “Biobert: 用于生物医学文本挖掘的预训练生物医学语言表示模型”, *Bioinformatics*, 卷。36, 没有。4, 第 1234–1240 页, 2020 年。[5] D. Support, “DeepL 翻译器的下一代语言模型”, <https://support.deepl.com/hc/en-us/articles/14241705319580-Next-Generation-language-model-for-DeepL-Translator>
- [6] S. Schneider, A. Baevski, R. Collobert 和 M. Auli, “wav2vec: 语音识别的无监督预训练”, *arXiv preprint arXiv:1904.05862*, 2019。[7] J. Guo, K. Han, H. Wu, C. Xu, Y. Tang, C. Xu 和 Y. Wang, “Cmt: 卷积神经网络遇到视觉变压器” 2021。[8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, 和 I. Polosukhin, “你所需要的就是注意力”, *Advances in neural information processing systems*, 卷。2017 年 3 月 30 日。[9] L. De Angelis, F. Baglivo, G. Arzilli, G. P. Privitera, P. Ferragina, A. E. Tozzi 和 C. Rizzo, “Chatgpt 和大语言模型的兴起: 公共卫生领域新的人工智能驱动的信息流行病威胁”, *Frontiers in Public Health*, 卷。11, p. 1166120, 2023。
- [10] C. Huyen, “多式联运模型”, <https://short.url/multimodal>, 2023 年, 访问时间: 2024-05-25。
- [11] I. Beltagy, M. E. Peters 和 A. Cohan, “Longformer: 长文档转换器”, *arXiv preprint arXiv:2004.05150*, 2020 年。
- [12] B. Chen, T. Dao, E. Winsor, Z. Song, A. Rudra 和 C. Ré, “分散脑: 统一稀疏和低阶注意力”, *Advances in Neural Information Processing Systems*, 卷。34, 第 17 413–17 426 页, 2021 年。

- [13] M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang *et al.*, “Big bird: Transformers for longer sequences,” *Advances in neural information processing systems*, vol. 33, pp. 17 283–17 297, 2020.
- [14] Y. Tay, D. Bahri, L. Yang, D. Metzler, and D.-C. Juan, “Sparse sinkhorn attention,” in *International Conference on Machine Learning*. PMLR, 2020, pp. 9438–9447.
- [15] Y. Tay, M. Dehghani, S. Abnar, Y. Shen, D. Bahri, P. Pham, J. Rao, L. Yang, S. Ruder, and D. Metzler, “Long range arena: A benchmark for efficient transformers,” *arXiv preprint arXiv:2011.04006*, 2020.
- [16] I. CBRE Group, “North america data center trends h1 2023,” 2023, accessed: 2024-06-03. [Online]. Available: <https://www.cbre.com/insights/reports/north-america-data-center-trends-h1-2023>
- [17] R. Vardhman and G. Defensor, “15 crucial data center statistics to know in 2024,” 2024, accessed: 2024-06-03. [Online]. Available: <https://techjury.net/blog/data-center-statistics/>
- [18] 3rd Generation Partnership Project, “About 3gpp,” n.d., accessed: 2024-06-17. [Online]. Available: <https://www.3gpp.org/about-3gpp>
- [19] A. Mudvari, N. Makris, and L. Tassiulas, “ML-driven scaling of 5g cloud-native rans,” in *2021 IEEE Global Communications Conference (GLOBECOM)*, 2021, pp. 1–6.
- [20] D. D. Lieira, M. S. Quessada, A. L. Cristiani, and R. I. Meneguette, “Algorithm for 5g resource management optimization in edge computing,” *IEEE Latin America Transactions*, vol. 19, no. 10, pp. 1772–1780, 2021.
- [21] A. Mudvari, N. Makris, and L. Tassiulas, “Exploring ml methods for dynamic scaling of beyond 5g cloud-native rans,” in *ICC 2022-IEEE International Conference on Communications*. IEEE, 2022, pp. 2284–2289.
- [22] R. Cziva, C. Anagnostopoulos, and D. P. Pezaros, “Dynamic, latency-optimal vnf placement at the network edge,” in *IEEE INFOCOM 2018 - IEEE Conference on Computer Communications*, 2018, pp. 693–701.
- [23] P. Research, “Edge data center market size to surpass usd 60.01 bn by 2033,” 2024, accessed: 2024-06-03. [Online]. Available: <https://www.precedenceresearch.com/edge-data-center-market>
- [24] A. Mudvari, A. Vainio, I. Ofeidis, S. Tarkoma, and L. Tassiulas, “Adaptive compression-aware split learning and inference for enhanced network efficiency,” 2023.
- [25] A. E. Eshratifar, A. Esmaili, and M. Pedram, “Bottlenet: A deep learning architecture for intelligent mobile cloud computing services,” in *2019 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. IEEE, 2019, pp. 1–6.
- [26] H.-J. Jeong, I. Jeong, H.-J. Lee, and S.-M. Moon, “Computation offloading for machine learning web apps in the edge server environment,” in *2018 IEEE 38th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2018, pp. 1492–1499.
- [27] X. Yang, J. Sun, Y. Yao, J. Xie, and C. Wang, “Differentially private label protection in split learning,” *arXiv preprint arXiv:2203.02073*, 2022.
- [28] D. Yao, L. Xiang, H. Xu, H. Ye, and Y. Chen, “Privacy-preserving split learning via patch shuffling over transformers,” in *2022 IEEE International Conference on Data Mining (ICDM)*. IEEE, 2022, pp. 638–647.
- [29] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, “Neurosurgeon: Collaborative intelligence between the cloud and mobile edge,” *ACM SIGARCH Computer Architecture News*, vol. 45, no. 1, pp. 615–629, 2017.
- [30] M. Wadhwa, G. R. Gupta, A. Sahu, R. Saini, and V. Mittal, “Pfsl: Personalized & fair split learning with data & label privacy for thin clients,” in *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, 2023, pp. 377–390.
- [31] C. Thapa, M. A. P. Chamikara, and S. A. Camtepe, “Advancements of federated learning towards privacy preservation: From federated learning to split learning,” in *Federated Learning Systems*, ser. Studies in Computational Intelligence. Springer, 2021, vol. 965, pp. 79–109, published in June 2021.
- [32] M. G. Arivazhagan, V. Aggarwal, A. K. Singh, and S. Choudhary, “Federated learning with personalization layers,” *arXiv preprint arXiv:1912.00818*, 2019.
- [33] K. Poularakis, J. Llorca, A. M. Tulino, I. Taylor, and L. Tassiulas, “Service placement and request routing in mec networks with storage, computation, and communication constraints,” *IEEE/ACM Transactions on Networking*, vol. 28, no. 3, pp. 1047–1060, 2020.
- [34] K. Poularakis, J. Llorca, A. M. Tulino, and L. Tassiulas, “Approximation algorithms for data-intensive service chain embedding,” in *Proceedings of the Twenty-First International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing*, 2020, pp. 131–140.
- [35] S. Pasteris, S. Wang, M. Herbster, and T. He, “Service placement with provable guarantees in heterogeneous edge computing systems,” in *IEEE INFOCOM 2019-IEEE Conference on Computer Communications*. IEEE, 2019, pp. 514–522.
- [36] B. Sonkoly, J. Czentye, M. Szalay, B. Németh, and L. Toka, “Survey on placement methods in the edge and beyond,” *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2590–2629, 2021.
- [37] H. T. Malazi, S. R. Chaudhry, A. Kazmi, A. Palade, C. Cabrera, G. White, and S. Clarke, “Dynamic service placement in multi-access edge computing: A systematic literature review,” *IEEE Access*, 2022.
- [38] Q. Luo, S. Hu, C. Li, G. Li, and W. Shi, “Resource scheduling in edge computing: A survey,” *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2131–2165, 2021.
- [39] V. N. Murthy, V. Singh, T. Chen, R. Manmatha, and D. Comaniciu, “Deep decision network for multi-class image classification,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 2240–2248.
- [40] S. Han, H. Shen, M. Philipose, S. Agarwal, A. Wolman, and A. Krishnamurthy, “Mcdnn: An approximation-based execution framework for deep stream processing under resource constraints,” in *Proceedings of the 14th Annual International Conference on Mobile Systems, Applications, and Services*, 2016, pp. 123–136.
- [41] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding,” *arXiv preprint arXiv:1510.00149*, 2015.
- [42] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, “Xnor-net: Imagenet classification using binary convolutional neural networks,” in *Computer Vision-ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part IV*. Springer, 2016, pp. 525–542.
- [43] P. Molchanov, A. Mallya, S. Tyree, I. Frosio, and J. Kautz, “Importance estimation for neural network pruning,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 11 264–11 272.
- [44] M. A. Carreira-Perpinán and Y. Idelbayev, “learning-compression” algorithms for neural net pruning,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 8532–8541.
- [45] C. Zhao, B. Ni, J. Zhang, Q. Zhao, W. Zhang, and Q. Tian, “Variational convolutional neural network pruning,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 2780–2789.
- [46] H. Hu, R. Peng, Y.-W. Tai, and C.-K. Tang, “Network trimming: A data-driven neuron pruning approach towards efficient deep architectures,” *arXiv preprint arXiv:1607.03250*, 2016.
- [47] Y. He, X. Zhang, and J. Sun, “Channel pruning for accelerating very deep neural networks,” in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 1389–1397.
- [48] H. Peng, J. Wu, S. Chen, and J. Huang, “Collaborative channel pruning for deep networks,” in *International Conference on Machine Learning*. PMLR, 2019, pp. 5113–5122.
- [49] E. Samikwa, A. Di Maio, and T. Braun, “Ares: Adaptive resource-aware split learning for internet of things,” *Computer Networks*, vol. 218, p. 109380, 2022.
- [50] —, “Disnet: Distributed micro-split deep learning in heterogeneous dynamic iot,” *IEEE internet of things journal*, 2023.
- [51] L. He and S. Todorovic, “Destr: Object detection with split transformer,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 9377–9386.
- [52] S. Park, G. Kim, J. Kim, B. Kim, and J. C. Ye, “Federated split vision transformer for covid-19 cxr diagnosis using task-agnostic training,” *arXiv preprint arXiv:2111.01338*, 2021.
- [53] P. Patel, E. Choukse, C. Zhang, Í. Goiri, A. Shah, S. Maleki, and R. Bianchini, “Splitwise: Efficient generative llm inference using phase splitting,” *arXiv preprint arXiv:2311.18677*, 2023.
- [54] N. Kitaev, L. Kaiser, and A. Levskaya, “Reformer: The efficient transformer,” *arXiv preprint arXiv:2001.04451*, 2020.
- [55] A. Vyas, A. Katharopoulos, and F. Fleuret, “Fast transformers with clustered attention,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 21 665–21 674, 2020.

- [13] M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang *et al.*, “大鸟：用于较长序列的变形金刚”, *Advances in neural information processing systems*, 卷. 33, 第 17 283–17 297 页, 2020. [14] Y. Tay, D. Bahri, L. Yang, D. Metzler 和 D.-C. Juan, “稀疏沉角注意力”, *International Conference on Machine Learning*. PMLR, 2020 年, 第 943 8–9447 页. [15] Y. Tay, M. Dehghani, S. Abnar, Y. Shen, D. Bahri, P. Pham, J. Rao, L. Yang, S. Ruder 和 D. Metzler, “远程竞技场：高效变压器的基准”, *arXiv preprint arXiv:2011.04006*, 2020 年. [16] I. CBRE Group, “2023 年上半年北美数据中心趋势”, 2023 年, 访问时间: 2024 年 6 月 3 日. [在线的]. 网址: <https://www.cbre.com/insights/reports/north-america-data-center-trends-h1-2023> [17] R. Vardhman 和 G. Defensor, “2024 年需要了解的 15 个关键数据中心统计数据”, 2024 年, 访问时间: 2024-06-03. [在线的]. 可用: <https://techjury.net/blog/data-center-statistics/> [18] 第三代合作伙伴项目, “关于 3gpp”, 日期不详, 访问时间: 2024-06-17. [在线的]. 可用: <https://www.3gpp.org/about-3gpp> [19] A. Mudvari, N. Makris 和 L. Tassiulas, “ML 驱动的 5g 云原生运行扩展”, *2021 IEEE Global Communications Conference (GLOBECOM)*, 2021 年, 第 1-6 页. [20] D. D. Lelira, M. S. Quessada, A. L. Cristiani 和 R. I. M eneguet, “边缘计算中 5G 资源管理优化算法”, *IEEE Latin America Transactions*, 卷. 19, 没有. 10, 第 1772–1780 页, 2021 年. [21] A. Mudvari, N. Makris 和 L. Tassiulas, “探索超过 5g 云原生动态扩展的机器学习方法”, 参见 *ICC 2022-IEEE International Conference on Communications*. IEEE, 2022, 第 2284–2289 页. [22] R. Cziva, C. Anagnostopoulos 和 D. P. Pezaros, “网络边缘的动态、延迟最优 vnf 放置”, *IEEE INFOCOM 2018 - IEEE Conference on Computer Communications*, 2018 年, 第 693–701 页. [23] P. Research, “到 2033 年, 边缘数据中心市场规模将超过 600.1 亿美元”, 2024 年, 访问时间: 2024-06-03. [在线的]. 可用: <https://www.precedenceresearch.com/edge-data-center-market> [24] A. Mudvari, A. Vainio, I. Ofeidis, S. Tarkoma 和 L. Tassiulas, “自适应压缩感知分割学习和推理以增强网络效率”, 2023 年. [25] A. E. Eshratifar, A. Esmaili 和 M. Pedram, “瓶颈: A 智能移动云计算服务的深度学习架构”, 见 *2019 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. IEEE, 2019 年, 第 1-6 页. [26] H.-J. Jeong, I. Jeon g, H.-J. 李, 和 S.-M. Moon, “边缘服务器环境中机器学习 Web 应用程序的计算卸载”, *2018 IEEE 38th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2018 年, 第 1492–1499 页. [27] X. Yang, J. Sun, Y. Yao, J. Xie 和 C. Wang, “分割学习中的差异化私有标签保护”, *arXiv preprint arXiv:2203.02073*, 2022. [28] D. Yao, L. Shang, H. Xu, H. Ye 和 Y. Chen, “通过 Transformer 上的补丁洗牌实现隐私保护分割学习”, *2022 IEEE International Conference on Data Mining (ICDM)*. IEEE, 2022 年, 第 638–647 页. [29] Y. Kang, J. Hauswald, C. Gau, A. Rovinski, T. Mudge, J. Mars 和 L. Tang, “神经外科医生: 云与移动边缘之间的协作智能”, *ACM SIGARCH Computer Architecture News*, 卷. 45, 没有. 1, 第 615–629 页, 2017 年. [30] M. Wadhwa, G. R. Gupta, A. Sahu, R. Saini 和 V. Mittal, “Pfs1: 针对瘦客户端的数据和标签隐私的个性化和公平分割学习”, 参见 *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, 2023 年, 第 377–390 页. [31] C. Thapa, M. A. P. Chamikara 和 S. A. Camtepe, “联邦学习在隐私保护方面的进展: 从联邦学习到分裂学习”, *Federated Learning Systems*, ser. 计算智能研究. 施普林格, 2021 年, 卷. 965, 第 79-109 页, 2021 年 6 月出版. [32] M. G. Arivazhagan, V. Aggarwal, A. K. Singh 和 S. Choudhary, “具有个性化层的联邦学习”, *arXiv preprint arXiv:1912.00818*, 2019. [33] K. Poularakis, J. Llorca, A. M. Tulino, I. Taylor 和 L. Tassiulas, “具有存储、计算和通信约束的 mec 网络中的服务放置和请求路由”, *IEEE/ACM Transactions on Networking*, 卷. 28, 没有. 3, 第 1047–1060 页, 2020 年. [34] K. Poularakis, J. Llorca, A. M. Tulino 和 L. Tassiulas, “数据密集型服务链嵌入的近似算法”, in *Proceedings of the Twenty-First International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing*, 2020 年, 第 131–140 页. [35] S. Pasteris, S. Wang, M. Herbster 和 T. He, “异构边缘计算系统中具有可证明保证的服务放置”, 参见 *IEEE INFOCOM 2019-IEEE Conference on Computer Communications*. IE EE, 2019 年, 第 514–522 页. [36] B. Sonkoly, J. Czentye, M. Szalay, B. Németh 和 L. Toka, “边缘及其他位置放置方法的调查”, *IEEE Communications Surveys & Tutorials*, 卷. 23, 没有. 4, 第 2590–2629 页, 2021 年. [37] H. T. Malazi, S. R. Chaudhry, A. Kazmi, A. Pala de, C. Cabrera, G. White 和 S. Clarke, “多访问边缘计算中的动态服务布局: 系统文献综述”, *IEEE Access*, 2022 年. [38] Q. Luo, S. Hu, C. Li, G. Li 和 W. Shi, “边缘计算中的资源调度: 一项调查”, *IEEE Communications Surveys & Tutorials*, 卷. 23, 没有. 4, 第 2131–2165 页, 2021 年. [39] V. N. Murthy, V. Singh, T. Chen, R. Manmatha 和 D. Comaniciu, “用于多类图像分类的深度决策网络”, *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016 年, 第 2240–2248 页. [40] S. Han, H. Shen, M. Philipose, S. Agarwal, A. Wolman 和 A. Krishnamurthy, “Mcdnn: 资源约束下基于近似的深度流处理执行框架”, *Proceedings of the 14th Annual International Conference on Mobile Systems, Applications, and Services*, 2016 年, 第 123-136 页. [41] S. Han, H. Mao 和 W. J. Dally, “深度压缩: 通过修剪、训练量化和霍夫曼编码压缩深度神经网络”, *arXiv preprint arXiv:1510.00149*, 2015. [42] M. Rastegari, V. Ordonez, J. Redmon 和 A. Farhadi, “Xnor-net: 使用二进制卷积神经网络进行 ImageNet 分类”, *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part IV*. 施普林格, 2016 年, 第 525–542 页. [43] P. Molchanov, A. Mallya, S. Tyree, I. Frosio 和 J. Kautz, “神经网络剪枝的重要性估计”, 载于 *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019 年, 第 11264–11272 页. [44] M. A. Carreira-Perpinán 和 Y. Idelbayev, “神经网络修剪的“学习压缩”算法”, *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018 年, 第 8532–8541 页. [45] C. Zhao, B. Ni, J. Zhang, Q. Zhao, W. Zhang, Q. Tian, “变分卷积神经网络剪枝”, 载于 *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019 年, 第 2780–2789 页. [46] 胡浩, 彭瑞, Y.-W. 泰和 C.-K. Tang, “网络修剪: 一种实现高效深层架构的数据驱动神经元修剪方法”, *arXiv preprint arXiv:1607.03250*, 2016. [47] Y. He, X. Zhang 和 J. Sun, “用于加速极深神经网络的通道修剪”, *Proceedings of the IEEE international conference on computer vision*, 2017 年, 第 1389–1397 页. [48] H. Peng, J. Wu, S. Chen 和 J. Huang, “深度网络的协作通道修剪”, 参见 *International Conference on Machine Learning*. PMLR, 2019 年, 第 5113–5122 页. [49] E. Samikwa, A. Di Maio 和 T. Braun, “Ares: 物联网的自适应资源感知分割学习”, *Computer Networks*, 卷. 218, p. 109380, 2022. [50] —, “Disnet: 异构动态物联网中的分布式微分割深度学习”, *IEEE internet of things journal*, 2023. [51] L. He 和 S. Todorovic, “Destr: 使用分割变压器进行对象检测”, *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022 年, 第 9377–9386 页. [52] S. Park, G. Kim, J. Kim, B. Kim 和 J. C. Ye, “使用与任务无关的训练进行 covid-19 cxr 诊断的联合分裂视觉变压器”, *arXiv preprint arXiv:2111.01338*, 2021. G oiri, A. Shah, S. Maleki 和 R. Bianchini, “Splitwise: 使用相分裂的高效生成 llm 推理”, *arXiv preprint arXiv:2311.18677*, 2023. [54] N. Kita ev, ☒. Kaiser 和 A. Levskaya, “改革者: 高效的转换器”, *arXiv preprint arXiv:2001.04451*, 2020 年. [55] A. Vyas, A. Katharopoulos 和 F. Fleuret, “具有集群注意力的快速转换器”, *Advances in Neural Information Processing Systems*, 卷. 33, 第 21665–21674 页, 2020 年.

- [56] K. Choromanski, V. Likhoshesterov, D. Dohan, X. Song, A. Gane, T. Sarlos, P. Hawkins, J. Davis, A. Mohiuddin, L. Kaiser *et al.*, “Rethinking attention with performers,” *arXiv preprint arXiv:2009.14794*, 2020.
- [57] S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma, “Linformer: Self-attention with linear complexity,” *arXiv preprint arXiv:2006.04768*, 2020.
- [58] F. C. V. Team, “fvcore: Collection of common code for fair computer vision projects,” <https://github.com/facebookresearch/fvcore>, accessed: 2024-06-16.
- [59] L. D. P. Pugliese and F. Guerriero, “A survey of resource constrained shortest path problems: Exact solution approaches,” *Networks*, vol. 62, no. 3, pp. 183–200, 2013.
- [60] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, “Albert: A lite bert for self-supervised learning of language representations,” *arXiv preprint arXiv:1909.11942*, 2019.
- [61] J. Shao and J. Zhang, “Bottlenet++: An end-to-end approach for feature compression in device-edge co-inference systems,” in *2020 IEEE International Conference on Communications Workshops (ICC Workshops)*. IEEE, 2020, pp. 1–6.
- [62] L. Shi, Z. Xu, Y. Sun, Y. Shi, Y. Fan, and X. Ding, “A dnn inference acceleration algorithm combining model partition and task allocation in heterogeneous edge computing system,” *Peer-to-Peer Networking and Applications*, vol. 14, no. 6, pp. 4031–4045, 2021.
- [63] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “Imagenet: A large-scale hierarchical image database,” in *CVPR09*, 2009.

[56] K. Choromanski, V. Likhoshesterov, D. Dohan, X. Song, A. Gane, T. S. S. Sarlos, P. Hawkins, J. Davis, A. Mohiuddin, L. Kaiser *et al.*, “重新思考表演者的注意力”, *arXiv preprint arXiv:2009.14794*, 2020. [57] S. Wang, B. Z. Li, M. Khabsa, H. Fang 和 H. Ma, “Linformer: 具有线性复杂性的自我关注”, *arXiv preprint arXiv:2006.04768*, 2020 年。[58] F. C. V. 团队, “fvcore: 公平计算机视觉项目的通用代码集合”, <https://github.com/facebookresearch/fvcore>, 访问时间: 2024-06-16. [59] L. D. P. Pugliese 和 F. Guerriero, “资源受限最短路径问题的调查: 精确解决方法”, *Networks*, 卷。62, 没有。3, 第 183–200 页, 2013 年。[60] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, “Albert: A lite bert for self-supervised Learning of language representations,” *arXiv preprint arXiv:1909.11942*, 2019. [61] J. Shao 和 J. Zhu, “Bottlenet++: An设备边缘协同推理系统中特征压缩的端到端方法”, 参见 *2020 IEEE International Conference on Communications Workshops (ICC Workshops)*. IEEE, 2020 年, 第 1-6 页。[62] 石丽, 徐正, 孙宇, 石宇, 范宇, 丁晓, “一种异构边缘计算系统中模型划分与任务分配相结合的DNN推理加速算法”, *Peer-to-Peer Networking and Applications*, 第1卷。14, 没有。6, 第 4031–4045 页, 2021 年。[63] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li 和 L. Fei-Fei, “Imagenet: 大规模分层图像数据库”, *CVPR09*, 2009 年。